



BookWise

Serverless Book Review Platform on AWS

AWS Cloud Computing Final Project

Course Name: AWS Cloud Computing

Instructor: Mr Anteneh

Group Members: Biruktawit Mikiyas, Kumneger Bekele

Submission Date: 7/17/2026

Contents

1	Introduction	3
2	Project Objectives	4
3	System Architecture	5
4	AWS Services Used	7
5	Project Implementation	8
5.1	Amazon S3 Implementation	8
5.1.1	S3 Bucket Creation	8
5.1.2	Frontend Folder Structure	9
5.1.3	Static Website Hosting	10
5.1.4	Bucket Policy Configuration	10
5.1.5	Frontend Deployment	11
5.1.6	Summary	11
5.2	IAM Implementation	11
5.2.1	IAM Groups	11
5.2.2	IAM Users	12
5.2.3	Custom Policy	12
5.3	Amazon API Gateway Implementation	13
5.3.1	REST API Design	13
5.3.2	API Resources and Methods	13
5.3.3	Integration with AWS Lambda	14
5.3.4	Request Flow	14
5.3.5	Security Considerations	14
5.3.6	Scalability and Availability	15
5.3.7	Summary	15
5.4	AWS Lambda	15
5.4.1	Overview	15
5.4.2	Role of Lambda in BookWise Architecture	15
5.4.3	Lambda Functions Implemented	16
5.4.4	Lambda Execution Role and Permissions	17
5.4.5	Lambda Integration with API Gateway	17
5.4.6	Monitoring Lambda with CloudWatch	17
5.4.7	Summary	18
5.5	Amazon DynamoDB	18
5.5.1	Overview	18
5.5.2	Role of DynamoDB in BookWise Architecture	18

5.5.3	DynamoDB Tables Created	19
5.5.4	DynamoDB Operations	20
5.5.5	Security and Access Control	20
5.5.6	Summary	20
5.6	Security Implementation	20
5.6.1	Overview	20
5.6.2	Identity and Access Management (IAM)	21
5.6.3	IAM Users	21
5.6.4	Least Privilege Access	21
5.6.5	S3 Bucket Security	22
5.6.6	Lambda Execution Role Security	22
5.6.7	Authentication Security Using Cognito	22
5.6.8	Summary	22
5.7	Testing	22
5.7.1	Overview	22
5.7.2	Frontend Testing	23
5.7.3	API Gateway Testing	23
5.7.4	Lambda and DynamoDB Testing	23
5.7.5	Authentication Testing	23
5.7.6	Monitoring Testing	24
5.7.7	Testing Summary	24
6	References	25
.1	Source Code Files	25
.1.1	Frontend Files	25
.1.2	JavaScript Files	37
.1.3	CSS Files	54
.1.4	Project Configuration Files	63

Chapter 1

Introduction

BookWise is a serverless cloud-based book review platform designed to allow users to browse books, authenticate securely, and submit reviews.

The project demonstrates the implementation of a modern cloud-native application using Amazon Web Services (AWS).

The traditional approach of managing servers introduces challenges such as maintenance, scalability, and infrastructure management. Therefore, a serverless architecture was selected because AWS automatically manages servers, allowing developers to focus on application functionality.

The main advantages of using AWS include:

- High scalability
- Reduced infrastructure management
- Improved reliability
- Pay-as-you-use pricing model
- Built-in security services

Chapter 2

Project Objectives

The objectives of the BookWise project are:

- Build a cloud-native web application.
- Host the frontend using Amazon S3.
- Develop REST APIs using Amazon API Gateway.
- Process backend requests using AWS Lambda.
- Store application data using DynamoDB.
- Implement authentication using Amazon Cognito.
- Monitor application activity using CloudWatch.
- Apply AWS security best practices using IAM.

Chapter 3

System Architecture

The overall architecture of BookWise follows a serverless AWS design.

Architecture

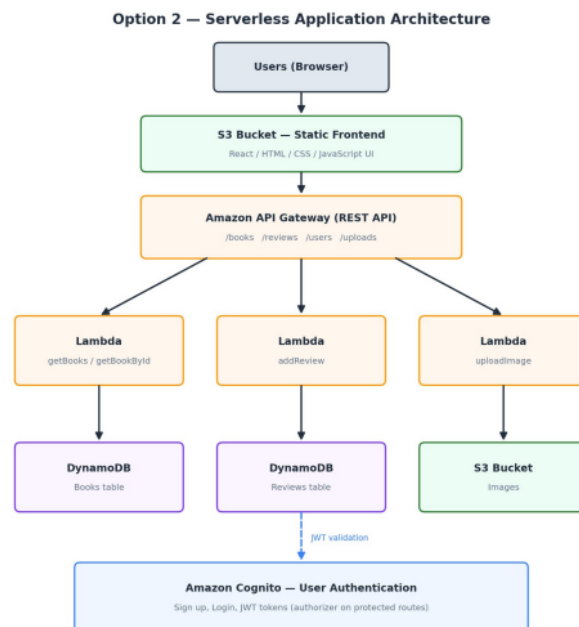
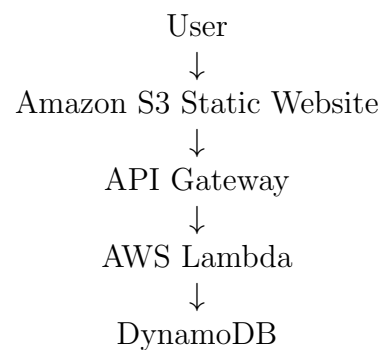


Figure 3.1: BookWise AWS Architecture Diagram

The application workflow is:



↓
CloudWatch

Architecture Explanation

- **User to S3:** Users access the frontend website hosted on Amazon S3.
- **S3 to API Gateway:** The frontend sends HTTP requests to backend APIs.
- **API Gateway to Lambda:** API Gateway receives requests and triggers Lambda functions.
- **Lambda to DynamoDB:** Lambda processes business logic and stores/retrieves book information.
- **CloudWatch Monitoring:** CloudWatch collects logs and metrics from AWS services.

Chapter 4

AWS Services Used

AWS Service	Purpose
Amazon S3	Static website hosting
API Gateway	REST API management
AWS Lambda	Backend processing logic
Amazon DynamoDB	Storage of books and reviews
Amazon Cognito	User authentication
Amazon CloudWatch	Monitoring and logging
IAM	Security and permissions management

Table 4.1: AWS Services Used

Chapter 5

Project Implementation

This chapter presents the practical implementation of the BookWise serverless web application. The project was developed using several Amazon Web Services (AWS) that work together to provide a secure, scalable, and highly available cloud solution. Each AWS service performs a specific responsibility within the architecture. The frontend application is hosted on Amazon S3 as a static website, API Gateway exposes REST endpoints, AWS Lambda executes the backend business logic, Amazon DynamoDB stores application data, Amazon Cognito manages user authentication, while Amazon CloudWatch provides monitoring and logging capabilities. Identity and Access Management (IAM) is used throughout the project to implement secure access control based on the principle of least privilege.

The implementation process followed the architecture presented in Chapter 3. Each service was configured individually before integrating them into a complete serverless application.

5.1 Amazon S3 Implementation

Amazon Simple Storage Service (Amazon S3) was selected to host the frontend of the BookWise application. Since the frontend consists of HTML, CSS, JavaScript, and image files, S3 provides a cost-effective and highly available platform for static website hosting without requiring a dedicated web server.

Unlike traditional web hosting, Amazon S3 automatically manages storage infrastructure, replication, durability, and scalability. This allows the development team to focus on application development instead of server administration.

The S3 bucket was configured using the AWS Management Console in the **US East (N. Virginia)** region (**us-east-1**), which is also the region used by the remaining AWS services in the project.

5.1.1 S3 Bucket Creation

The first implementation step involved creating an S3 bucket dedicated to hosting the frontend website.

The following configuration was used during bucket creation.

Table 5.1: Amazon S3 Bucket Configuration

Configuration	Value
Bucket Name	bookreview-2026-qiyas-project1
AWS Region	us-east-1 (N. Virginia)
Bucket Type	General Purpose
Object Ownership	Bucket Owner Enforced (ACL Disabled)
Block Public Access	Disabled
Versioning	Disabled
Encryption	SSE-S3 (Default Encryption)
Object Lock	Disabled

The bucket name was selected to ensure uniqueness within AWS while reflecting the purpose of the project. Server-side encryption (SSE-S3) was enabled to automatically encrypt all uploaded objects without requiring additional configuration.

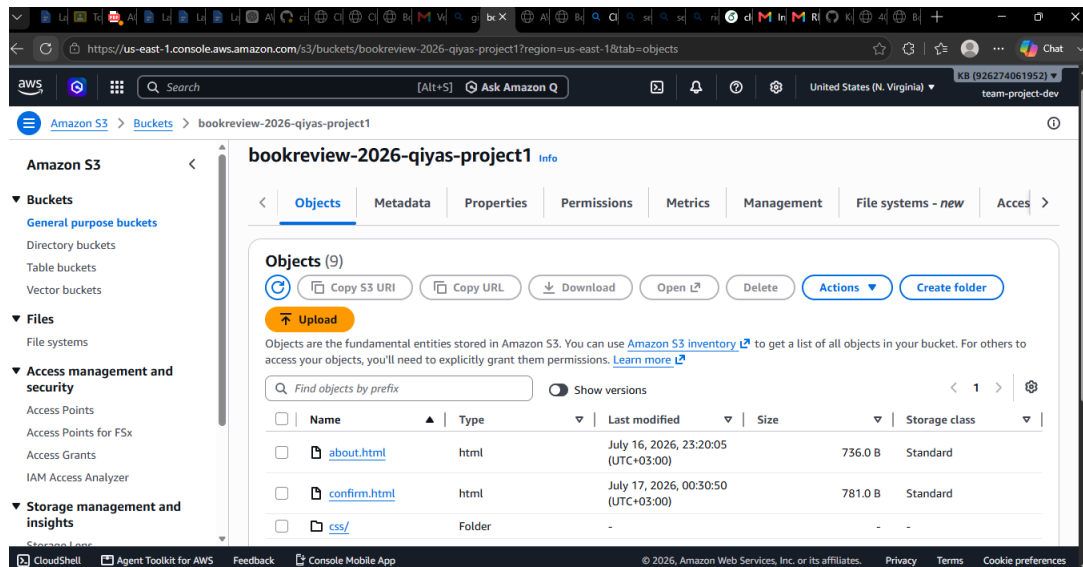


Figure 5.1: Amazon S3 Bucket Configuration

5.1.2 Frontend Folder Structure

To improve maintainability and simplify future integration with backend services, the frontend files were organized into separate directories. This structure follows common web development practices and makes the application easier to extend as additional features are implemented.

The frontend project consists of HTML pages, cascading style sheets, JavaScript files, and image assets.

Table 5.2: Frontend Project Structure

book-review-frontend/
index.html
about.html
error.html login.html
confirm.html
register.html
css/style.css
js/script.js
auth.js
js/api.js
images/
README.md

After development was completed locally, all frontend resources were uploaded into the S3 bucket while preserving the directory structure.

5.1.3 Static Website Hosting

Amazon S3 provides native support for hosting static websites. Static website hosting was enabled for the project bucket after uploading all frontend files.

The following configuration was applied.

Table 5.3: Static Website Hosting Configuration

Property	Value
Hosting	Enabled
Index Document	index.html
Error Document	error.html

After enabling static website hosting, Amazon S3 generated a public website endpoint. This endpoint allows users to access the frontend application directly through a web browser without requiring a traditional web server.

5.1.4 Bucket Policy Configuration

Since the frontend website must be accessible by internet users, a bucket policy was created to allow public read access to website objects.

Rather than making the entire AWS account publicly accessible, the policy grants read permission only to objects stored inside the project bucket. This approach follows AWS recommendations for static website hosting while minimizing unnecessary permissions.

During implementation, an initial policy configuration produced the error *"Policy has invalid resource."* This issue occurred because the bucket ARN did not exactly match the bucket name. After correcting the resource ARN, the bucket policy was successfully applied.

5.1.5 Frontend Deployment

After configuring static website hosting and applying the bucket policy, the BookWise frontend became publicly accessible through the S3 website endpoint.

The homepage was successfully displayed, including navigation links, responsive design, category cards, and user interface enhancements. Additional improvements such as an About page, dark mode support, and improved frontend organization were implemented to provide a more professional appearance than the default project implementation.

Future integration with Amazon API Gateway and AWS Lambda will allow the static frontend to communicate dynamically with backend services.

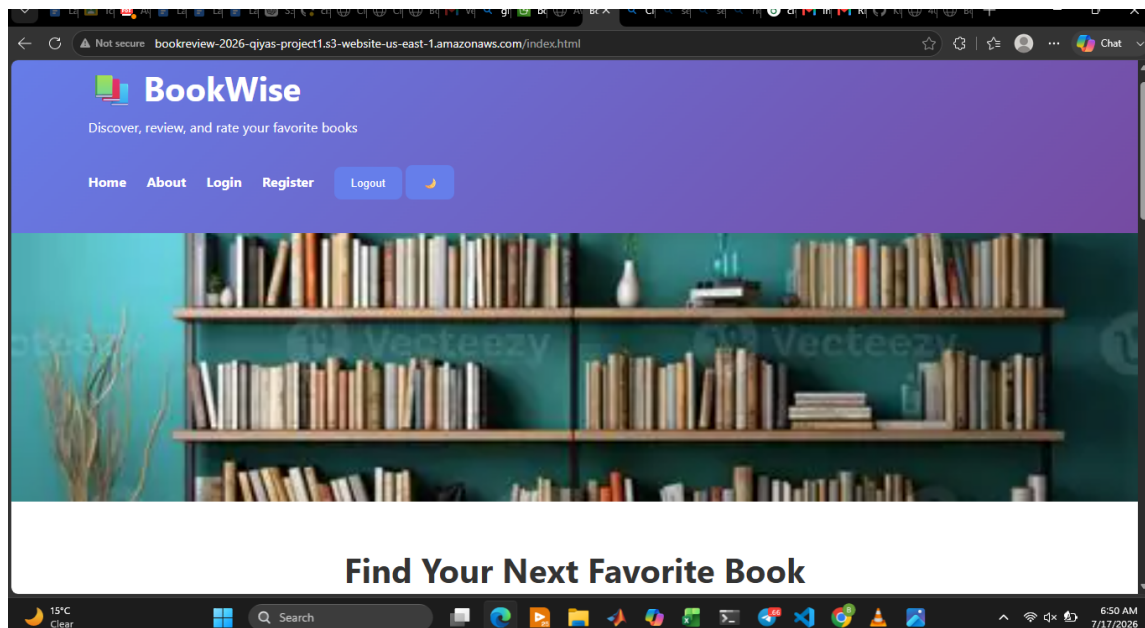


Figure 5.2: BookWise Homepage Hosted on Amazon S3

5.1.6 Summary

Amazon S3 provided a reliable platform for hosting the frontend application while eliminating the need for server management. By combining static website hosting, public object access through bucket policies, and an organized frontend directory structure, the BookWise website became scalable, highly available, and ready for integration with the remaining serverless AWS services.

5.2 IAM Implementation

IAM was used to manage access control and follow the principle of least privilege.

5.2.1 IAM Groups

BookWise-Developers

Permissions:

- API Gateway Administrator

- AWSLambda Full Access
- BookWise-S3-Frontend-Access

BookWise-Operations

Permissions:

- AmazonS3ReadOnlyAccess
- CloudWatchFullAccessV2

5.2.2 IAM Users

```
bookwise-developer
|
|
BookWise-Developers
```

```
bookwise-operations
|
|
BookWise-Operations
```

5.2.3 Custom Policy

A custom IAM policy was created instead of providing unrestricted permissions. Developers were granted access only to the required frontend S3 bucket.

This follows AWS least privilege security principles.

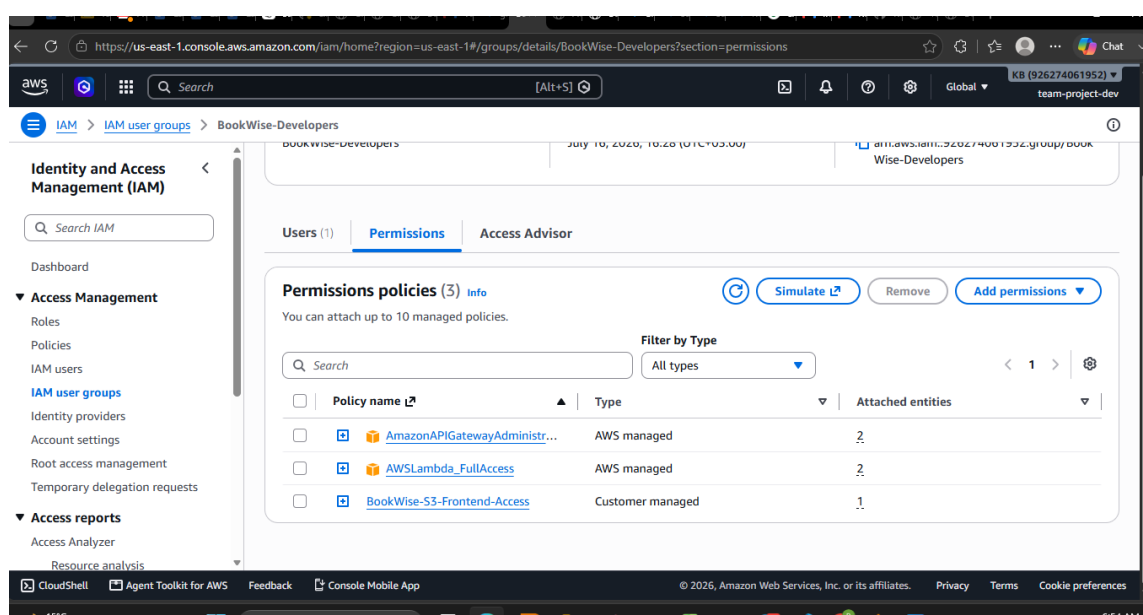


Figure 5.3: IAM user groups.png

5.3 Amazon API Gateway Implementation

Amazon API Gateway was implemented as the communication layer between the static frontend hosted on Amazon S3 and the backend services running on AWS Lambda. Instead of allowing the frontend application to access backend resources directly, API Gateway exposes a secure RESTful API that receives HTTP requests, validates them, and forwards them to the appropriate Lambda function.

Using API Gateway provides several advantages, including request routing, centralized API management, scalability, security, monitoring, and seamless integration with other AWS services. Since API Gateway is a fully managed service, no server infrastructure is required, allowing the application to automatically scale based on incoming user requests.

Within the BookWise application, API Gateway acts as the entry point for all client interactions. When a user accesses the frontend website and performs an operation such as retrieving books or submitting a review, the request is first sent to API Gateway. The gateway then invokes the corresponding Lambda function, which processes the request and communicates with Amazon DynamoDB before returning the response to the client.

This design follows the serverless architecture recommended by AWS and enables each application component to operate independently while maintaining secure communication between services.

5.3.1 REST API Design

The REST API was designed following common REST architectural principles, where resources are accessed through HTTP methods such as GET and POST. Each endpoint represents a specific application resource, allowing frontend components to communicate with backend services using standard web protocols.

The BookWise API consists of endpoints responsible for retrieving available books, obtaining detailed information about a selected book, and allowing authenticated users to submit reviews.

Table 5.4 summarizes the primary REST endpoints implemented within the application.

Table 5.4: BookWise REST API Endpoints

Endpoint	Method	Purpose
/books	GET	Retrieve all available books.
/books/{bookId}	GET	Retrieve detailed information for a specific book.
/reviews	POST	Submit a new book review.

Each endpoint is connected to an AWS Lambda function that performs the required business logic before interacting with Amazon DynamoDB.

5.3.2 API Resources and Methods

Amazon API Gateway organizes APIs into resources and methods. A resource represents a specific URI path, while methods define the HTTP operations that clients may perform on that resource.

For the BookWise application, multiple resources were created to separate application functionality into logical components. This modular design improves maintainability and simplifies future expansion of the application.

HTTP GET methods were used to retrieve data without modifying database contents, while HTTP POST methods were implemented for creating new reviews submitted by authenticated users.

5.3.3 Integration with AWS Lambda

Rather than processing requests directly, API Gateway was configured to invoke AWS Lambda functions using Lambda Proxy Integration. This integration allows the complete HTTP request—including headers, query parameters, request body, and path parameters—to be forwarded directly to the Lambda function.

After processing the request, Lambda returns a JSON response containing the appropriate HTTP status code and response body. API Gateway then forwards this response back to the frontend application.

This integration minimizes backend complexity while allowing Lambda functions to remain independent from frontend implementation details.

5.3.4 Request Flow

The communication between the frontend and backend follows a sequential workflow.

1. A user interacts with the BookWise frontend hosted on Amazon S3.
2. JavaScript sends an HTTP request to Amazon API Gateway.
3. API Gateway validates the request and routes it to the appropriate AWS Lambda function.
4. The Lambda function executes the required business logic.
5. Lambda retrieves or stores data within Amazon DynamoDB.
6. The processed response is returned to API Gateway.
7. API Gateway returns the JSON response to the frontend application.
8. The frontend updates the user interface using the received data.

This workflow separates presentation, business logic, and data storage into independent components, making the application easier to maintain and scale.

5.3.5 Security Considerations

Security was an important consideration during the API design process. Rather than allowing direct database access from the client application, API Gateway serves as the only public entry point into the backend system.

User authentication is performed through Amazon Cognito before protected operations such as submitting reviews are allowed. API Gateway works together with Cognito to ensure that only authenticated users may invoke secured endpoints.

In addition, IAM permissions were configured to ensure that only authorized AWS services are able to invoke backend Lambda functions, reducing the overall attack surface of the application.

5.3.6 Scalability and Availability

One of the major advantages of Amazon API Gateway is its ability to automatically scale according to incoming traffic. Since API Gateway is a fully managed AWS service, there is no need to provision or maintain web servers.

As the number of users increases, API Gateway automatically handles request routing while invoking multiple Lambda functions in parallel when necessary. This elastic scaling capability enables the BookWise application to accommodate changing workloads without requiring manual infrastructure management.

5.3.7 Summary

Amazon API Gateway provides the communication layer that connects the static frontend with the serverless backend. Through RESTful API endpoints, secure Lambda integration, automatic scalability, and seamless interaction with AWS services, API Gateway enables reliable and efficient communication throughout the BookWise application. Its integration with AWS Lambda, Amazon DynamoDB, and Amazon Cognito forms a key component of the overall serverless architecture.

5.4 AWS Lambda

5.4.1 Overview

Amazon Web Services (AWS) Lambda is a serverless compute service that allows developers to run backend code without managing servers. Lambda automatically manages the underlying infrastructure, including server provisioning, scaling, and availability. The service executes code only when triggered by an event, which makes it a cost-effective solution for applications with variable workloads.

In the BookWise platform, AWS Lambda was used to implement the backend business logic of the application. Instead of maintaining a traditional server, Lambda functions process requests received from Amazon API Gateway and communicate with Amazon DynamoDB to retrieve and store book and review data.

5.4.2 Role of Lambda in BookWise Architecture

The Lambda functions act as the processing layer between the frontend application and the database. When a user interacts with the BookWise web application, the request follows the serverless workflow:

**User → Amazon S3 Frontend → API Gateway → AWS Lambda →
DynamoDB**

API Gateway receives HTTP requests from the frontend and triggers the appropriate Lambda function. The Lambda function processes the request, performs the required

database operation in DynamoDB, and returns the response back through API Gateway to the user interface.

5.4.3 Lambda Functions Implemented

Three Lambda functions were created for the BookWise application:

getBooks Function

The **getBooks** Lambda function is responsible for retrieving all available books from the DynamoDB **Books** table. It uses the DynamoDB Scan operation to collect book information and returns the results as a JSON response.

The function is triggered by the following API endpoint:

GET /books

The returned data includes information such as:

- Book ID
- Book title
- Author name
- Book description

getBookById Function

The **getBookById** Lambda function retrieves details about a specific book using its unique identifier. The function receives the book ID from the API request path and queries the DynamoDB table using the **GetItem** operation.

The function is accessed through:

GET /books/{bookId}

This allows users to request detailed information about an individual book.

addReview Function

The **addReview** Lambda function handles user review submissions. When a user submits a rating and comment, the function validates the request data and stores the review information in the DynamoDB **Reviews** table.

The function is triggered by:

POST /reviews

Each review contains:

- Book ID
- User ID
- Rating value
- Review comment
- Creation timestamp

5.4.4 Lambda Execution Role and Permissions

Each Lambda function runs using an IAM execution role named:

`BookWiseLambdaRole`

The role follows the principle of least privilege by granting only the permissions required for application operation.

The Lambda role includes permissions for:

- Reading book information from DynamoDB
- Writing review data to DynamoDB
- Creating CloudWatch log groups
- Publishing execution logs to CloudWatch

This ensures that Lambda functions can access required AWS resources while maintaining application security.

5.4.5 Lambda Integration with API Gateway

Amazon API Gateway was configured as the trigger for the Lambda functions. Each API route was connected to a specific Lambda function using Lambda proxy integration.

The implemented API routes are shown below:

HTTP Method	API Route	Lambda Function
GET	/books	getBooks
GET	/books/{bookId}	getBookById
POST	/reviews	addReview

Table 5.5: Lambda Functions Connected to API Gateway

5.4.6 Monitoring Lambda with CloudWatch

AWS Lambda automatically publishes performance metrics to Amazon CloudWatch. A CloudWatch dashboard was created for monitoring the health and performance of the BookWise backend.

The monitored Lambda metrics include:

- **Invocations:** Number of times Lambda functions are executed.
- **Errors:** Number of failed Lambda executions.
- **Duration:** Average execution time of Lambda functions.

These metrics help identify performance issues, failures, and application usage patterns.

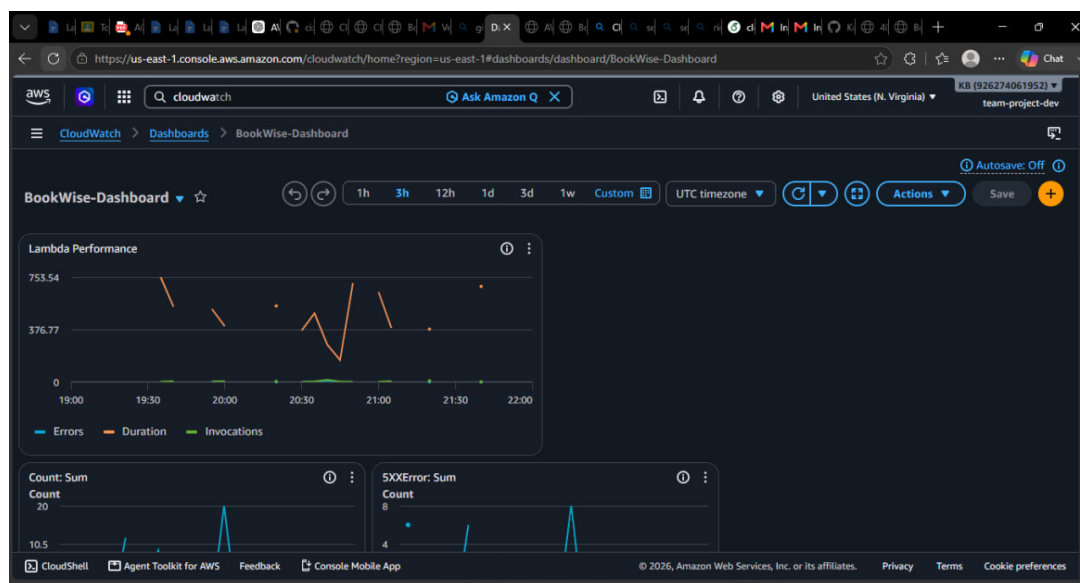


Figure 5.4: AWS CloudWatch Dashboard Showing Lambda Performance Metrics

5.4.7 Summary

AWS Lambda provided the backend compute layer for the BookWise serverless application. By using Lambda functions with API Gateway and DynamoDB, the application achieved a scalable architecture without requiring traditional server management. Lambda automatically handles execution and scaling, while CloudWatch provides monitoring and visibility into backend performance.

5.5 Amazon DynamoDB

5.5.1 Overview

Amazon DynamoDB is a fully managed NoSQL database service provided by AWS. It provides fast and predictable performance with automatic scaling, removing the need for database server management. DynamoDB stores data in tables consisting of items and attributes, making it suitable for applications that require flexible data structures and high availability.

In the BookWise platform, DynamoDB was used as the primary database service for storing information about books and user reviews. The serverless database design allows the application to scale automatically while maintaining low operational overhead.

5.5.2 Role of DynamoDB in BookWise Architecture

DynamoDB acts as the data storage layer of the BookWise serverless architecture. Lambda functions communicate with DynamoDB to retrieve book information and store user-generated reviews.

The data flow of the application is:

Frontend Application → API Gateway → Lambda Functions → DynamoDB Tables

When a user requests book information, the request is processed by Lambda, which queries DynamoDB and returns the required data through API Gateway. When a user submits a review, Lambda validates the information and stores it in the Reviews table.

5.5.3 DynamoDB Tables Created

Two DynamoDB tables were created for the BookWise application:

Books Table

The **Books** table stores information about available books in the platform.

The table uses:

- Partition key: `bookId`
- Billing mode: Pay-per-request

Each book item contains attributes such as:

- Book ID
- Title
- Author
- Description

The table is accessed by the `getBooks` and `getBookById` Lambda functions.

Reviews Table

The **Reviews** table stores user reviews and ratings for books.

The table uses:

- Partition key: `bookId`
- Sort key: `reviewId`
- Billing mode: Pay-per-request

Each review item contains:

- Book ID
- Review ID
- User ID
- Rating
- Comment
- Creation timestamp

The `addReview` Lambda function writes new reviews into this table.

5.5.4 DynamoDB Operations

The BookWise application uses different DynamoDB operations through AWS SDK:

- **Scan:** Used by the `getBooks` function to retrieve all available books.
- **GetItem:** Used by the `getBookById` function to retrieve a specific book.
- **PutItem:** Used by the `addReview` function to store new reviews.

These operations allow the application to efficiently manage book and review data.

5.5.5 Security and Access Control

Access to DynamoDB was controlled using AWS Identity and Access Management (IAM). Lambda functions use an execution role with specific DynamoDB permissions following the principle of least privilege.

The permissions granted include:

- `dynamodb:GetItem`
- `dynamodb:Scan`
- `dynamodb:Query`
- `dynamodb:PutItem`
- `dynamodb:UpdateItem`
- `dynamodb>DeleteItem`

This ensures that only authorized services can access or modify application data.

5.5.6 Summary

Amazon DynamoDB provided a scalable and serverless database solution for the BookWise application. By storing books and reviews in DynamoDB tables and accessing them through Lambda functions, the platform achieved a flexible architecture with automatic scaling and minimal database administration.

5.6 Security Implementation

5.6.1 Overview

Security is an essential part of the BookWise serverless architecture. AWS security services were used to protect application resources, control user access, and ensure that only authorized users and services can interact with the system.

The security implementation of BookWise follows the AWS shared responsibility model and applies the principle of least privilege, where users and services receive only the permissions required to perform their tasks.

5.6.2 Identity and Access Management (IAM)

AWS Identity and Access Management (IAM) was used to control access to AWS resources. IAM users, groups, and policies were configured to manage permissions securely.

The following IAM groups were created:

BookWise-Developers Group

The **BookWise-Developers** group was created for users responsible for application development and management.

The group permissions include:

- Amazon API Gateway administration permissions
- AWS Lambda management permissions
- Access to the BookWise frontend S3 bucket through a custom policy

BookWise-Operations Group

The **BookWise-Operations** group was created for monitoring and operational tasks.

The group permissions include:

- Amazon S3 read-only access
- CloudWatch monitoring permissions

5.6.3 IAM Users

Two IAM users were created and assigned to their respective groups:

- **bookwise-developer** was assigned to the BookWise-Developers group.
- **bookwise-operations** was assigned to the BookWise-Operations group.

This approach avoids sharing administrator credentials and provides controlled access based on user responsibilities.

5.6.4 Least Privilege Access

A custom IAM policy was created to restrict access to only the required S3 frontend bucket.

Instead of granting unrestricted access to all Amazon S3 resources, developers were allowed to access only the BookWise project bucket.

This follows the AWS security best practice of least privilege, reducing the risk of accidental or unauthorized access.

5.6.5 S3 Bucket Security

The BookWise frontend was hosted using Amazon S3 static website hosting.

Security measures included:

- Configuring a bucket policy to control website access.
- Restricting resource access using IAM permissions.
- Separating frontend resources from backend services.

5.6.6 Lambda Execution Role Security

Lambda functions were configured with an IAM execution role.

The execution role provides Lambda with only the required permissions to:

- Read and write DynamoDB data.
- Create CloudWatch log streams.
- Publish execution logs.

This prevents unnecessary permissions from being assigned to backend services.

5.6.7 Authentication Security Using Cognito

Amazon Cognito was used to secure user access to the BookWise platform.

Security features include:

- User registration and authentication.
- Email verification.
- Secure password management.
- Token-based authentication using JWT tokens.

5.6.8 Summary

The BookWise application implements multiple AWS security mechanisms including IAM access control, least privilege policies, Cognito authentication, and secure Lambda permissions. These security practices protect application resources while maintaining controlled access for users and administrators.

5.7 Testing

5.7.1 Overview

Testing was performed to verify that all components of the BookWise serverless application were functioning correctly. The testing process focused on validating the frontend interface, AWS service integration, API communication, authentication, and database operations.

The application was tested after integrating Amazon S3, API Gateway, AWS Lambda, DynamoDB, Amazon Cognito, and CloudWatch.

5.7.2 Frontend Testing

The BookWise frontend was deployed using Amazon S3 static website hosting.

The following frontend functionalities were tested:

- Website loading through the S3 website endpoint.
- Navigation between Home and About pages.
- Displaying featured books and dynamically loaded books.
- Searching for books using the search functionality.
- Responsive layout and user interface behavior.

5.7.3 API Gateway Testing

The REST API endpoints were tested to verify communication between the frontend and backend services.

The following API routes were tested:

HTTP Method	Endpoint	Result
GET	/books	Successfully retrieved books
GET	/books/{bookId}	Successfully retrieved specific book data
POST	/reviews	Successfully submitted reviews

Table 5.6: API Gateway Testing Results

5.7.4 Lambda and DynamoDB Testing

The Lambda functions were tested to ensure correct interaction with DynamoDB tables.

The following operations were verified:

- Retrieving book information from the Books table.
- Retrieving individual book details using a book ID.
- Adding new reviews into the Reviews table.
- Handling invalid requests and missing input fields.

5.7.5 Authentication Testing

Amazon Cognito authentication was tested through the BookWise login and registration pages.

The tests included:

- Creating a new user account.
- Verifying user registration.
- Logging in using valid credentials.
- Generating authentication tokens after successful login.
- Logging out and clearing authentication information.

5.7.6 Monitoring Testing

CloudWatch monitoring was verified by checking the availability of:

- Lambda invocation metrics.
- Lambda execution duration.
- Lambda error metrics.
- API Gateway request metrics.
- API Gateway latency and error metrics.

5.7.7 Testing Summary

The testing process confirmed that the BookWise platform successfully integrates AWS serverless services. The frontend communicates correctly with API Gateway, Lambda functions process requests successfully, DynamoDB stores application data, and Cognito provides user authentication.

Chapter 6

References

- [AWS Documentation](#)
- [Amazon S3 Documentation](#)
- [Amazon API Gateway Documentation](#)
- [AWS Lambda Documentation](#)
- [Amazon DynamoDB Documentation](#)
- [Amazon Cognito Documentation](#)
- [Amazon CloudWatch Documentation](#)

.1 Source Code Files

This appendix contains the main source code files used in the BookWise serverless application. The following listings provide placeholders for the frontend, backend integration, authentication, and configuration files.

.1.1 Frontend Files

index.html

Listing 1: BookWise Home Page (index.html)

```
<!DOCTYPE html>
<html>

<head>

<meta charset="UTF-8">

<title>BookWise - Book Reviews</title>

<link rel="stylesheet" href="css/style.css">

</head>
```

```
<body>

<header>

<div class="container">

<h1>          BookWise</h1>

<p>
Discover , review , and rate your favorite books
</p>

<nav>

<a href="index.html">Home</a>

<a href="about.html">About</a>

<a href="login.html">Login</a>

<a href="register.html">Register</a>

<button onclick="logoutUser()">
Logout
</button>

<button onclick="toggleDarkMode()">

</button>

</nav>

</div>

</header>

<section class="hero">

<div class="hero-image">
```

```


</div>

<div class="container">

  <h2>
  Find Your Next Favorite Book
  </h2>

  <p>
  Join a community of readers sharing honest reviews.
  </p>

  <button onclick="scrollBooks()">
  Explore Books
  </button>

</div>

</section>

<section class="container">

  <h2>
  Book Categories
  </h2>

  <div class="categories">

    <div>      Fiction</div>

    <div>      Technology</div>
```

```
<div>          Self Development</div>

<div>          Science</div>

</div>


<h2>
Featured Books
</h2>


<div class="featured-books">


<div class="book-card">



<h3>
The Alchemist
</h3>

<p>
A journey of dreams and discovery.
</p>

</div>


<div class="book-card">



<h3>
Clean Code
</h3>

<p>
Learn software development practices.
```

```
</p>

</div>


<div class="book-card">



<h3>
Rich Dad Poor Dad
</h3>

<p>
A book about financial education and mindset.
</p>

</div>


</div>


<h2 id="books-section">
Browse Books
</h2>


<div class="search-container">

<input

id="search-bar"

type="text"

placeholder="Search books..."

onkeyup="searchBooks()">
```

```
</div>

<div id="books-grid" class="books-grid">

</div>

<div
id="review-section"
style="display:none;">

<h2>
Add Your Review
</h2>

<div class="review-form">

<input

id="rating"

type="number"

min="1"

max="5"

placeholder="Rating 1-5">

<textarea

id="comment"
```

```
placeholder="Write your review">

</textarea>


<button onclick="submitReview()">
Submit Review
</button>


</div>


</div>


</section>


<footer>

<p>
BookWise Serverless Book Review Platform
</p>

</footer>


<!-- AWS SDK -->

<script src="https://sdk.amazonaws.com/js/aws-sdk-2.1691.0.min.js
"></script>
```



```
<!-- Amazon Cognito Identity SDK -->

<script src="https://cdn.jsdelivr.net/npm/amazon-cognito-identity
  -js@3.0.14/dist/amazon-cognito-identity.min.js"></script>


<!-- Project JavaScript -->

<script src="js/auth.js"></script>

<script src="js/api.js"></script>

<script src="js/script.js"></script>


</body>

</html>.
```

about.html

```
<html>
  <head>
    <meta charset="UTF-8">
    <title>About BookWise</title>
    <link rel="stylesheet" href="css/style.css">
  </head>
  <body>
    <header>
      <div class="container">
        <h1>About BookWise</h1>
      </div>
    </header>
    <div class="container">
      <h2>Our Project</h2>
      <p>
        BookWise is a serverless book review platform built using AWS cloud services.
      </p>
      <br>
      <h3>AWS Technologies</h3>
      <ul>
        <li>Amazon S3 - Frontend Hosting</li>
```

```
|li|API Gateway - API Management|/li|
|li|AWS Lambda - Backend Processing|/li|
|li|DynamoDB - Database|/li|
|li|Cognito - Authentication|/li|
|li|CloudWatch - Monitoring|/li|
|/ul|
|/div|
|/body|
|/html|.
```

register.html

Listing 2: User Registration Page (register.html)

```
<!DOCTYPE html>
<html>

<head>

    <meta charset="UTF-8">

    <title>BookWise Register</title>

    <link rel="stylesheet" href="css/style.css">

</head>

<body>

    <div class="container">

        <h2>Create Account</h2>

        <input
            id="email"
            type="email"
            placeholder="Email">

        <br><br>

        <input
            id="password"
            type="password"
            placeholder="Password">

        <br><br>

        <button onclick="registerUser()">
            Register
        </button>
```

```
<p>
    Already have an account?
    <a href="login.html">
        Login
    </a>
</p>

</div>

<!-- AWS SDK -->
<script src="https://sdk.amazonaws.com/js/aws-sdk-2.1691.0.
    min.js"></script>

<!-- Amazon Cognito Identity SDK -->
<script src="https://cdn.jsdelivr.net/npm/amazon-cognito-
    identity-js@3.0.14/dist/amazon-cognito-identity.min.js"></
    script>

<!-- Authentication Script -->
<script src="js/auth.js"></script>

</body>

</html>
```

login.html

Listing 3: User Login Page (login.html)

```
<!DOCTYPE html>
<html>

<head>

    <meta charset="UTF-8">

    <title>BookWise Login</title>

    <link rel="stylesheet" href="css/style.css">

</head>

<body>

    <div class="container">

        <h2>Login</h2>

        <input
            id="email"
            type="email"
```

```
        placeholder="Email">

    <br><br>

    <input
        id="password"
        type="password"
        placeholder="Password">

    <br><br>

    <button onclick="loginUser()">
        Login
    </button>

    <p>
        Don't have an account?
        <a href="register.html">
            Register
        </a>
    </p>

</div>

<!-- AWS SDK -->
<script src="https://sdk.amazonaws.com/js/aws-sdk-2.1691.0.
    min.js"></script>

<!-- Amazon Cognito Identity JS -->
<script src="https://cdn.jsdelivr.net/npm/amazon-cognito-
    identity-js@3.0.14/dist/amazon-cognito-identity.min.js"></
    script>

<!-- Your authentication code -->
<script src="js/auth.js"></script>

</body>

</html>.
```

confirm.html

Listing 4: Cognito Confirmation Page (confirm.html)

```
Insert confirm.html source code here.
```

error.html

Listing 5: Error Page (error.html)

```
<!DOCTYPE html>
<html>

<head>

<title>Confirm Account - BookWise</title>

<link rel="stylesheet" href="css/style.css">

</head>

<body>

<div class="container">

<h2>Verify Your Account</h2>

<p>
Enter the verification code sent to your email.
</p>

<input
id="email"
type="email"
placeholder="Email">

<br><br>

<input
id="code"
type="text"
placeholder="Verification Code">

<br><br>

<button onclick="confirmUser()">
Verify Account
</button>

</div>
```

```
<script src="https://sdk.amazonaws.com/js/aws-sdk-2.1691.0.min.js"
"></script>

<script src="https://cdn.jsdelivr.net/npm/amazon-cognito-identity
-js@3.0.14/dist/amazon-cognito-identity.min.js"></script>

<script src="js/auth.js"></script>

</body>

</html>.
```

.1.2 JavaScript Files

api.js

Listing 6: API Integration File (api.js)

```
const API_URL =
"https://iwm7t3c4id.execute-api.us-east-1.amazonaws.com/prod";

let currentBookId = null;

let currentUser = localStorage.getItem("user");

let allBooks = [];

// Convert DynamoDB format if needed
function convertDynamoDBItem(item){

    return {

        bookId: item.bookId?.S || item.bookId,

        title: item.title?.S || item.title,

        author: item.author?.S || item.author,

        description: item.description?.S || item.description

    };

}

// Browse Books cover images
function getBookImage(bookId){
```

```
    const images = {  
      "book-001": "serverless-AWS.jpg",  
      "book-002": "cloud computing made simple.jpg",  
      "book-003": "mastering AWS Lambda.jpg"  
    };  
  
    return images[bookId] || "serverless-AWS.jpg";  
  }  
}
```

```
// Load books from DynamoDB through API Gateway  
async function loadBooks(){
```

```
  try{  
  
    const response = await fetch(  
      `${API_URL}/books`  
    );  
  
    let books = await response.json();  
  
    // Handle DynamoDB response format  
    books = books.map(  
      book => convertDynamoDBItem(book)  
    );  
  
    allBooks = books;  
  
    const grid =  
    document.getElementById("books-grid");  
  
    if(!grid){  
      console.log("books-grid not found");  
    }  
  }  
}
```

```
        return;
    }

    grid.innerHTML = books.map(book => `

    <div class="book-card">

        <h3>
        ${book.title}
        </h3>

        <p>
        <strong>Author:</strong>
        ${book.author}
        </p>

        <p>
        ${book.description}
        </p>

        <button onclick="openBook('${book.bookId}')">

        Add Review

        </button>

    </div>

    `).join("");
```



```
    }

    catch(error){

        console.error(
            "Error loading books:",
            error
        );

    }

}

// Get single book
async function getBook(bookId){

    try{

        const response = await fetch(

            `${API_URL}/books/${bookId}`

        );

        let book =
            await response.json();

        return convertDynamoDBItem(book);

    }

    catch(error){
```

```
        console.error(  
            "Error getting book:",  
            error  
        );  
    }  
}  
  
// Submit Review  
async function submitReview(){  
  
    const review = {  
  
        bookId: currentBookId,  
  
        userId: currentUser,  
  
        rating:  
        document.getElementById("rating").value,  
  
        comment:  
        document.getElementById("comment").value  
  
    };  
  
    try{  
  
        const response = await fetch(  
            `${API_URL}/reviews`,  
            {  
                method: "POST",  
  
                headers: {
```

```
        "Content-Type":
        "application/json"

    },

    body:
    JSON.stringify(review)

}

);

const result =
await response.json();

alert(

    result.message ||
    "Review submitted successfully"

);

}

catch(error){

    console.error(
        "Error submitting review:",
        error
    );

}

}
```

```
// Open selected book
async function openBook(bookId){

    currentBookId = bookId;

    const book =
    await getBook(bookId);

    console.log(
        "Selected book:",
        book
    );

    const reviewSection =
    document.getElementById(
        "review-section"
    );

    reviewSection.style.display =
    "block";

    reviewSection.scrollIntoView({

        behavior:"smooth"

    });

}

// Start loading books
loadBooks();

function searchBooks(){

    const searchValue =
```

```
document.getElementById("search-bar")
.value
.toLowerCase();

const filteredBooks =
allBooks.filter(book =>

    book.title
    .toLowerCase()
    .includes(searchValue)

    ||

    book.author
    .toLowerCase()
    .includes(searchValue)

);

const grid =
document.getElementById("books-grid");

grid.innerHTML =
filteredBooks.map(book => '

    <div class="book-card">

        <h3>
        ${book.title}
        </h3>

        <p>
        <strong>Author:</strong>
        ${book.author}
        </p>
```

```
        <p>
        ${book.description}
        </p>

        <button onclick="openBook('${book.bookId}')">
        Add Review
        </button>

    </div>

    ').join("");
}.

```

auth.js

Listing 7: Amazon Cognito Authentication File (auth.js)

```
// =====
// Cognito Configuration
// =====

const USER_POOL_ID = "us-east-1_6IfUSLyTk";

const CLIENT_ID = "3eh4kmh71ev2i324mkuiiqrsoa";

// Check Cognito SDK loaded
if (typeof AmazonCognitoIdentity === "undefined") {
    alert("Cognito SDK not loaded. Check your script imports.");
}

// Create User Pool
const poolData = {
    UserPoolId: USER_POOL_ID,
    ClientId: CLIENT_ID
};

```

```
const userPool =
new AmazonCognitoIdentity.CognitoUserPool(poolData);

// =====
// REGISTER USER
// =====

function registerUser(){

    const email =
document.getElementById("email").value;

    const password =
document.getElementById("password").value;

    const attributeList = [];

    const emailAttribute =
new AmazonCognitoIdentity.CognitoUserAttribute({

        Name: "email",

        Value: email

    });

    attributeList.push(emailAttribute);

    userPool.signUp(

        email,

        password,

        attributeList,
```

```
        null ,

        function(error,result){

            if(error){

                alert(error.message);

                return;

            }

            alert(
"Registration successful. Check your email for the verification
    code."
);

window.location.href="confirm.html";

        }

    );

}

// =====
// LOGIN USER
// =====

function loginUser(){

    const email =

    document.getElementById("email").value;
```



```
const password =  
document.getElementById("password").value;  
  
const authenticationData = {  
    Username: email,  
    Password: password  
};  
  
const authenticationDetails =  
new AmazonCognitoIdentity.AuthenticationDetails(  
    authenticationData  
);  
  
const userData = {  
    Username: email,  
    Pool: userPool  
};  
  
const cognitoUser =  
new AmazonCognitoIdentity.CognitoUser(userData);  
  
cognitoUser.authenticateUser(  
    authenticationDetails,  
  
    {  
  
        onSuccess: function(result){
```

```
        const idToken =

        result
        .getIdToken()
        .getJwtToken();

        localStorage.setItem(

            "token",

            idToken

        );

        localStorage.setItem(

            "user",

            email

        );

        alert(
            "Login successful"
        );

        window.location.href="index.html";

    },

    onFailure:function(error){

        alert(

            "Login failed: " + error.message
```

```
        );

    }

    }

);

}

// =====
// LOGOUT USER
// =====

function logoutUser(){

    localStorage.removeItem("token");

    localStorage.removeItem("user");

    alert(
        "You have been logged out"
    );

    window.location.href="login.html";

}
```

```
// =====  
// CHECK CURRENT USER  
// =====  
  
function getCurrentUser(){  
  
    return localStorage.getItem("user");  
  
}  
function confirmUser(){  
  
    const email =  
    document.getElementById("email").value;  
  
    const code =  
    document.getElementById("code").value;  
  
    const userData = {  
  
        Username: email,  
  
        Pool: userPool  
    };  
  
    const cognitoUser =  
    new AmazonCognitoIdentity.CognitoUser(userData);  
  
    cognitoUser.confirmRegistration(  
  
        code,  
  
        true,  
  
        function(error,result){  
  
            if(error){  
  
                alert(error.message);  
  
            }  
        }  
    );  
}
```

```
        return;

    }

    alert(
        "Account verified successfully"
    );

    window.location.href="login.html";

}

);

}
```

script.js

Listing 8: Frontend Script File (script.js)

```
// =====
// Protect BookWise Application
// =====

function checkLogin(){

    const user = localStorage.getItem("user");
    const token = localStorage.getItem("token");

    // If user is not authenticated,
    // redirect to login page

    if(!user || !token){

        window.location.href = "login.html";

    }

}

// Run login check only on index.html

if(window.location.pathname.endsWith("index.html")
|| window.location.pathname === "/" ){

    checkLogin();

}
```

```
}

// =====
// Dark Mode
// =====

function toggleDarkMode(){

    document.body.classList.toggle("dark");
}

// =====
// Scroll to Books Section
// =====

function scrollBooks(){

    document
    .getElementById("books-section")
    .scrollIntoView({

        behavior:"smooth"

    });
}

// =====
// Login Placeholder
// =====

function showLogin(){

    window.location.href = "login.html";
}

// =====
// Signup Placeholder
// =====
```

```
function showSignup(){  
  
    window.location.href = "register.html";  
  
}.
```

index.js

Listing 9: Lambda Function Source Code (index.js)

```
import { DynamoDBClient, ScanCommand } from "@aws-sdk/client-dynamodb";  
import { DynamoDBDocumentClient } from "@aws-sdk/lib-dynamodb";  
  
const client = new DynamoDBClient({});  
const docClient = DynamoDBDocumentClient.from(client);  
  
export const handler = async (event) => {  
    try {  
        const command = new ScanCommand({  
            TableName: "Books"  
        });  
        const response = await docClient.send(command);  
  
        return {  
            statusCode: 200,  
            headers: {  
                "Content-Type": "application/json",  
                "Access-Control-Allow-Origin": "*"  
            },  
            body: JSON.stringify(response.Items)  
        };  
    } catch (error) {  
        console.error("Error:", error);  
        return {  
            statusCode: 500,  
            headers: {  
                "Access-Control-Allow-Origin": "*"  
            },  
            body: JSON.stringify({ error: "Failed to fetch books" })  
        };  
    }  
};
```

.1.3 CSS Files

style.css

Listing 10: Website Styling File (style.css)

```
* {  
    margin: 0;  
    padding: 0;  
    box-sizing: border-box;  
}  
  
body {  
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;  
    background: #f0f2f5;  
    color: #333;  
    transition: 0.3s;  
}  
  
.container {  
    max-width: 1200px;  
    margin: auto;  
    padding: 20px;  
}  
  
/* Header */  
  
header {  
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%)  
        ;  
    color: white;  
    padding: 20px 0;  
}  
  
header h1 {  
    font-size: 2.5em;  
}  
  
header p {  
    margin-top: 10px;  
}  
  
/* Navigation */  
  
nav {  
    margin-top: 20px;  
}
```



```
nav a {
    color: white;
    text-decoration: none;
    margin-right: 20px;
    font-weight: bold;
}

nav a:hover {
    text-decoration: underline;
}

/* Buttons */

button {

    background: #667eea;
    color: white;

    border: none;

    padding: 12px 20px;

    border-radius: 8px;

    cursor: pointer;

    margin-top: 15px;
}

button:hover {

    background: #5a67d8;
}

/* Hero Section */

.hero {

    text-align: center;

    background: white;

    padding-bottom: 40px;
```

```
}

.hero-image img {

    width: 100%;

    height: 320px;

    object-fit: cover;

}

.hero h2 {

    font-size: 40px;

    margin-top: 30px;

}

.hero p {

    font-size: 18px;

    margin-top: 15px;

}

/* Categories */

.categories {

    display: grid;

    grid-template-columns: repeat(4,1fr);

    gap:20px;

    margin:30px 0;

}
```

```
.categories div {  
    background:white;  
    padding:25px;  
    border-radius:10px;  
    text-align:center;  
    box-shadow:0 2px 10px rgba(0,0,0,0.1);  
    font-size:18px;  
}  
  
/* Books loaded from API */  
  
.books-grid {  
    display:grid;  
    grid-template-columns: repeat(auto-fit,minmax(250px,1fr));  
    gap:20px;  
    margin:30px 0;  
}  
  
.book-card {  
    background:white;  
    border-radius:10px;  
    padding:20px;  
    box-shadow:0 2px 10px rgba(0,0,0,0.1);  
    cursor:pointer;  
    transition:0.3s;  
}
```

```
.book-card:hover {  
    transform:translateY(-5px);  
}  
  
/* NEW: API book cover images */  
.books-grid .book-card img {  
    width:180px;  
    height:250px;  
    object-fit:cover;  
    border-radius:10px;  
    display:block;  
    margin:0 auto 15px;  
    cursor:pointer;  
}  
  
.book-card h3 {  
    color:#333;  
    margin-bottom:10px;  
}  
  
.book-card p {  
    margin-top:10px;  
    color:#666;  
}
```

```
/* Featured Books */

.featured-books {

    display:grid;

    grid-template-columns:repeat(auto-fit,minmax(280px,1fr));

    gap:30px;

    margin:30px 0;

}

/* Featured Books - Larger Covers */

.featured-books .book-card img {

    width:100%;

    height:330px;

    object-fit:cover;

    border-radius:10px;

}

/* Review Section */

.review-form {

    background:white;

    padding:20px;

    border-radius:10px;

    margin-top:20px;

}

.review-form input,

.review-form textarea {

    width:100%;
```

```
padding:12px;

margin:10px 0;

border:1px solid #ddd;

border-radius:5px;
}

.review-form textarea {

    height:120px;
}

/* Footer */
footer {

    text-align:center;

    background:#222;

    color:white;

    padding:20px;

    margin-top:50px;
}

/* Dark Mode */
.dark {

    background:#121212;

    color:white;
}
```

```
.dark header {  
    background:linear-gradient(135deg,#232526,#414345);  
}  
  
.dark .hero,  
.dark .book-card,  
.dark .categories div,  
.dark .review-form {  
    background:#1e1e1e;  
    color:white;  
}  
  
.dark .book-card p {  
    color:#ddd;  
}  
  
/* Responsive Design */  
@media(max-width:800px){  
    .categories {  
        grid-template-columns:repeat(2,1fr);  
    }  
  
    .hero h2 {  
        font-size:30px;  
    }  
}
```

```
@media(max-width:500px){

    .categories {

        grid-template-columns:1fr;

    }

    nav a {

        display:block;

        margin:10px 0;

    }

}

/* Search Bar */

.search-container{

    text-align:center;

    margin:30px 0;

}

#search-bar{

    width:60%;

    padding:15px;

    font-size:18px;

    border-radius:10px;

    border:1px solid #ccc;

}.

```

.1.4 Project Configuration Files

package.json

Listing 11: Node.js Package Configuration (package.json)

```

{
  "name": "getbooks",
  "version": "1.0.0",
  "lockfileVersion": 3,
  "requires": true,
  "packages": {
    "": {
      "name": "getbooks",
      "version": "1.0.0",
      "dependencies": {
        "@aws-sdk/client-dynamodb": "^3.0.0",
        "@aws-sdk/lib-dynamodb": "^3.0.0"
      }
    },
    "node_modules/@aws-sdk/client-dynamodb": {
      "version": "3.1088.0",
      "resolved": "https://registry.npmjs.org/@aws-sdk/client-dynamodb/-/client-dynamodb-3.1088.0.tgz",
      "integrity": "sha512-CVxp88YVq87tW9i2+y6/yjtmvVmVt9V6tz2XFBVC2T3zNG/LntIKS1131d321ZX9Bipc9VmKxhJ+1+JVb7m/dg==",
      "license": "Apache-2.0",
      "dependencies": {
        "@aws-sdk/core": "^3.975.3",
        "@aws-sdk/credential-provider-node": "^3.972.69",
        "@aws-sdk/dynamodb-protocol": "^3.973.33",
        "@aws-sdk/middleware-endpoint-discovery": "^3.972.25",
        "@aws-sdk/types": "^3.974.2",
        "@smithy/core": "^3.29.4",
        "@smithy/fetch-http-handler": "^5.6.6",
        "@smithy/node-http-handler": "^4.9.6",
        "@smithy/types": "^4.16.1",
        "tslib": "^2.6.2"
      },
      "engines": {
        "node": ">=20.0.0"
      }
    },
    "node_modules/@aws-sdk/core": {
      "version": "3.975.3",
      "resolved": "https://registry.npmjs.org/@aws-sdk/core/-/core-3.975.3.tgz",
      "integrity": "sha512-ur3kCKuvPLqlsZ2XlvnNBVQ7KkpSu6Y6d0TwSPHLrFpTEfZM8isLBJc4cg96WB7Gif==",
      "license": "Apache-2.0",
      "dependencies": {
        "@aws-sdk/types": "^3.974.2",
        "@aws-sdk/xml-builder": "^3.972.36",
        "@aws/lambda-invoke-store": "^0.3.0",

```

```

    "@smithy/core": "^3.29.4",
    "@smithy/signature-v4": "^5.6.5",
    "@smithy/types": "^4.16.1",
    "browser": "^2.11.0",
    "tslib": "^2.6.2"
  },
  "engines": {
    "node": ">=20.0.0"
  }
},
"node_modules/@aws-sdk/credential-provider-env": {
  "version": "3.972.59",
  "resolved": "https://registry.npmjs.org/@aws-sdk/credential-provider-env/-/credential-provider-env-3.972.59.tgz",
  "integrity": "sha512-Ny5e4Mfh3QPmiAc0AiUe+cbTXDlxkU3Rc+EpW0fyWeWEy6yp7Fa1KmfNeCc+1a8by9zQ9gtohmiQUkMPScF3ng==",
  "license": "Apache-2.0",
  "dependencies": {
    "@aws-sdk/core": "^3.975.3",
    "@aws-sdk/types": "^3.974.2",
    "@smithy/core": "^3.29.4",
    "@smithy/types": "^4.16.1",
    "tslib": "^2.6.2"
  },
  "engines": {
    "node": ">=20.0.0"
  }
},
"node_modules/@aws-sdk/credential-provider-http": {
  "version": "3.972.61",
  "resolved": "https://registry.npmjs.org/@aws-sdk/credential-provider-http/-/credential-provider-http-3.972.61.tgz",
  "integrity": "sha512-8jAjqStl5Ytq4+HF3X/9f+EmRinaRbGRRtQGktlPfBRVx73H+R1y48vIeXerQtYGFaUqkEp3fT6jP854rV02yQ==",
  "license": "Apache-2.0",
  "dependencies": {
    "@aws-sdk/core": "^3.975.3",
    "@aws-sdk/types": "^3.974.2",
    "@smithy/core": "^3.29.4",
    "@smithy/fetch-http-handler": "^5.6.6",
    "@smithy/node-http-handler": "^4.9.6",
    "@smithy/types": "^4.16.1",
    "tslib": "^2.6.2"
  },
  "engines": {
    "node": ">=20.0.0"
  }
},
"node_modules/@aws-sdk/credential-provider-ini": {
  "version": "3.973.3",

```

```

"resolved": "https://registry.npmjs.org/@aws-sdk/credential
  -provider-ini/-/credential-provider-ini-3.973.3.tgz",
"integrity": "sha512-WpuqYX4gGkx++fCTSWE8+41
  JzkZVcrI50SH48Ml4CsG1pyuHKyMmpw/FixBHDrmjoQ553PmeCLa/
  fZlCst+WyA==",
"license": "Apache-2.0",
"dependencies": {
  "@aws-sdk/core": "^3.975.3",
  "@aws-sdk/credential-provider-env": "^3.972.59",
  "@aws-sdk/credential-provider-http": "^3.972.61",
  "@aws-sdk/credential-provider-login": "^3.972.65",
  "@aws-sdk/credential-provider-process": "^3.972.59",
  "@aws-sdk/credential-provider-sso": "^3.973.3",
  "@aws-sdk/credential-provider-web-identity": "^3.972.65",
  "@aws-sdk/nested-clients": "^3.997.33",
  "@aws-sdk/types": "^3.974.2",
  "@smithy/core": "^3.29.4",
  "@smithy/credential-provider-imds": "^4.4.9",
  "@smithy/types": "^4.16.1",
  "tslib": "^2.6.2"
},
"engines": {
  "node": ">=20.0.0"
}
},
"node_modules/@aws-sdk/credential-provider-login": {
  "version": "3.972.65",
  "resolved": "https://registry.npmjs.org/@aws-sdk/credential
    -provider-login/-/credential-provider-login-3.972.65.tgz
    ",
  "integrity": "sha512-xr9rgjYEdmC2Tpg2lwt9o+n0EaK9Qpd+
    dBjzrVCuWWyQfvh091Ezu0Hh9ts2VUx0ZxmS/k5T9msa34e4R1bnrQ
    ==",
  "license": "Apache-2.0",
  "dependencies": {
    "@aws-sdk/core": "^3.975.3",
    "@aws-sdk/nested-clients": "^3.997.33",
    "@aws-sdk/types": "^3.974.2",
    "@smithy/core": "^3.29.4",
    "@smithy/types": "^4.16.1",
    "tslib": "^2.6.2"
  },
  "engines": {
    "node": ">=20.0.0"
  }
},
"node_modules/@aws-sdk/credential-provider-node": {
  "version": "3.972.69",
  "resolved": "https://registry.npmjs.org/@aws-sdk/credential
    -provider-node/-/credential-provider-node-3.972.69.tgz",

```

```

    "integrity": "sha512-wbJGGesd0Tl18bmUcbj1xJ+
      e7CpuRJ6PIpMywLFuUttGy615lua87cJ0EA8pFpY/
      QgPuUXbnupWBtSPJ9tyZhg==",
    "license": "Apache-2.0",
    "dependencies": {
      "@aws-sdk/credential-provider-env": "^3.972.59",
      "@aws-sdk/credential-provider-http": "^3.972.61",
      "@aws-sdk/credential-provider-ini": "^3.973.3",
      "@aws-sdk/credential-provider-process": "^3.972.59",
      "@aws-sdk/credential-provider-sso": "^3.973.3",
      "@aws-sdk/credential-provider-web-identity": "^3.972.65",
      "@aws-sdk/types": "^3.974.2",
      "@smithy/core": "^3.29.4",
      "@smithy/credential-provider-imds": "^4.4.9",
      "@smithy/types": "^4.16.1",
      "tslib": "^2.6.2"
    },
    "engines": {
      "node": ">=20.0.0"
    }
  },
  "node_modules/@aws-sdk/credential-provider-process": {
    "version": "3.972.59",
    "resolved": "https://registry.npmjs.org/@aws-sdk/credential
      -provider-process/-/credential-provider-process
      -3.972.59.tgz",
    "integrity": "sha512-DlZF2/MhLlatDdlrIy3CUCpfdBbLrKx+3SMjVo+
      WyHnPPwzkc/M3vwAHw40VJf7DMv0+4vfRqSCMc/E9I1auN0g==",
    "license": "Apache-2.0",
    "dependencies": {
      "@aws-sdk/core": "^3.975.3",
      "@aws-sdk/types": "^3.974.2",
      "@smithy/core": "^3.29.4",
      "@smithy/types": "^4.16.1",
      "tslib": "^2.6.2"
    },
    "engines": {
      "node": ">=20.0.0"
    }
  },
  "node_modules/@aws-sdk/credential-provider-sso": {
    "version": "3.973.3",
    "resolved": "https://registry.npmjs.org/@aws-sdk/credential
      -provider-sso/-/credential-provider-sso-3.973.3.tgz",
    "integrity": "sha512-
      hmdDHoy2G5Es2e8IgelNMYUuSQI6uCIAKZMJ2u2PdKDhxxvbk1uWD/g4+
      R7R5c/tJfKEB1+KjjWiaoCr/S+ZTiQ==",
    "license": "Apache-2.0",
    "dependencies": {
      "@aws-sdk/core": "^3.975.3",
      "@aws-sdk/nested-clients": "^3.997.33",

```

```

    "@aws-sdk/token-providers": "3.1088.0",
    "@aws-sdk/types": "^3.974.2",
    "@smithy/core": "^3.29.4",
    "@smithy/types": "^4.16.1",
    "tslib": "^2.6.2"
  },
  "engines": {
    "node": ">=20.0.0"
  }
},
"node_modules/@aws-sdk/credential-provider-web-identity": {
  "version": "3.972.65",
  "resolved": "https://registry.npmjs.org/@aws-sdk/credential-provider-web-identity/-/credential-provider-web-identity-3.972.65.tgz",
  "integrity": "sha512-gHQB/Kt0chjk/JQDa/GJDqmAvEuVn8n7z10wK2h0LFM9TUDRkohg004aEF+s2sBLM0br7Cl5W6P7phgjrrJvLQ==",
  "license": "Apache-2.0",
  "dependencies": {
    "@aws-sdk/core": "^3.975.3",
    "@aws-sdk/nested-clients": "^3.997.33",
    "@aws-sdk/types": "^3.974.2",
    "@smithy/core": "^3.29.4",
    "@smithy/types": "^4.16.1",
    "tslib": "^2.6.2"
  },
  "engines": {
    "node": ">=20.0.0"
  }
},
"node_modules/@aws-sdk/dynamodb-codec": {
  "version": "3.973.33",
  "resolved": "https://registry.npmjs.org/@aws-sdk/dynamodb-codec/-/dynamodb-codec-3.973.33.tgz",
  "integrity": "sha512-awY2mG1zvTy22j4ryZL8jfWQdcuppsw17Mxw1tjG92h0+52JoFZaaTWrJIQp066t/aj4JC4II29UJb6g644vA==",
  "license": "Apache-2.0",
  "dependencies": {
    "@aws-sdk/core": "^3.975.3",
    "@smithy/core": "^3.29.4",
    "@smithy/types": "^4.16.1",
    "tslib": "^2.6.2"
  },
  "engines": {
    "node": ">=20.0.0"
  }
},
"node_modules/@aws-sdk/endpoint-cache": {
  "version": "3.972.9",

```

```

    "resolved": "https://registry.npmjs.org/@aws-sdk/endpoint-cache/-/endpoint-cache-3.972.9.tgz",
    "integrity": "sha512-LFvdgq8SriaskUcjpBMDE7J2c9RmuT5v3gU36/zNv71EU5DKUis4FmGFjCMelKCCViFeVrQADBA1Ii0YRhEx6Q==",
    "license": "Apache-2.0",
    "dependencies": {
      "mnemonist": "0.38.3",
      "tslib": "^2.6.2"
    },
    "engines": {
      "node": ">=20.0.0"
    }
  },
  "node_modules/@aws-sdk/lib-dynamodb": {
    "version": "3.1088.0",
    "resolved": "https://registry.npmjs.org/@aws-sdk/lib-dynamodb/-/lib-dynamodb-3.1088.0.tgz",
    "integrity": "sha512-d57jl+BVQWiA1+5NRUtN3/ZKbn0ipQjNuimF3T2LCtA2IcYdCAw37ESDer39ldN4C8BxWRC44RbCmLCkmsx3+w==",
    "license": "Apache-2.0",
    "dependencies": {
      "@aws-sdk/core": "^3.975.3",
      "@aws-sdk/util-dynamodb": "^3.996.7",
      "@smithy/core": "^3.29.4",
      "@smithy/types": "^4.16.1",
      "tslib": "^2.6.2"
    },
    "engines": {
      "node": ">=20.0.0"
    },
    "peerDependencies": {
      "@aws-sdk/client-dynamodb": "^3.1088.0"
    }
  },
  "node_modules/@aws-sdk/middleware-endpoint-discovery": {
    "version": "3.972.25",
    "resolved": "https://registry.npmjs.org/@aws-sdk/middleware-endpoint-discovery/-/middleware-endpoint-discovery-3.972.25.tgz",
    "integrity": "sha512-5G2aVPmbWC5dFI76D0vsNg2L0fgWP4uybcetf+06dzeZuRtpihnow88f0l5zm3+ABdIw27FKgyFzV4yB5Bm29Q==",
    "license": "Apache-2.0",
    "dependencies": {
      "@aws-sdk/endpoint-cache": "^3.972.9",
      "@aws-sdk/types": "^3.974.2",
      "@smithy/core": "^3.29.4",
      "@smithy/types": "^4.16.1",
      "tslib": "^2.6.2"
    },
    "engines": {

```

```

    "node": ">=20.0.0"
  },
  "node_modules/@aws-sdk/nested-clients": {
    "version": "3.997.33",
    "resolved": "https://registry.npmjs.org/@aws-sdk/nested-clients/-/nested-clients-3.997.33.tgz",
    "integrity": "sha512-dVZ0roI/r3/ENvqNGgjMPul+jjLz9GddfVusgTXlVjffZj5isib0xecLkGQbRPp8X0uX+RAfjXLFGPkD1JS5xrw==",
    "license": "Apache-2.0",
    "dependencies": {
      "@aws-sdk/core": "^3.975.3",
      "@aws-sdk/signature-v4-multi-region": "^3.996.41",
      "@aws-sdk/types": "^3.974.2",
      "@smithy/core": "^3.29.4",
      "@smithy/fetch-http-handler": "^5.6.6",
      "@smithy/node-http-handler": "^4.9.6",
      "@smithy/types": "^4.16.1",
      "tslib": "^2.6.2"
    },
    "engines": {
      "node": ">=20.0.0"
    }
  },
  "node_modules/@aws-sdk/signature-v4-multi-region": {
    "version": "3.996.41",
    "resolved": "https://registry.npmjs.org/@aws-sdk/signature-v4-multi-region/-/signature-v4-multi-region-3.996.41.tgz",
    "integrity": "sha512-QMUytg+FQMGouc8gHS00KoYih3+N6cqmqVI/pQG0Io7Nr70pQaiXjSY0uL+vsPZ1tymY4LAQ8MYcHJmws5LRxng==",
    "license": "Apache-2.0",
    "dependencies": {
      "@aws-sdk/types": "^3.974.2",
      "@smithy/signature-v4": "^5.6.5",
      "@smithy/types": "^4.16.1",
      "tslib": "^2.6.2"
    },
    "engines": {
      "node": ">=20.0.0"
    }
  },
  "node_modules/@aws-sdk/token-providers": {
    "version": "3.1088.0",
    "resolved": "https://registry.npmjs.org/@aws-sdk/token-providers/-/token-providers-3.1088.0.tgz",
    "integrity": "sha512-4ObatWt2qpJg5FBk4L00KrTQYzaqeewAttd03r9Z081H9YqLtpTzLyIdyOmJ+nVdfY0nqISkKNfmzP22bNDhwyw==",
    "license": "Apache-2.0",

```

```

    "dependencies": {
      "@aws-sdk/core": "^3.975.3",
      "@aws-sdk/nested-clients": "^3.997.33",
      "@aws-sdk/types": "^3.974.2",
      "@smithy/core": "^3.29.4",
      "@smithy/types": "^4.16.1",
      "tslib": "^2.6.2"
    },
    "engines": {
      "node": ">=20.0.0"
    }
  },
  "node_modules/@aws-sdk/types": {
    "version": "3.974.2",
    "resolved": "https://registry.npmjs.org/@aws-sdk/types/-/types-3.974.2.tgz",
    "integrity": "sha512-W6IUtSxFbH6X7Wb7DzGCV5QiFQsd0g8b0fntpmDxQlZBoKWUMBu/JPQR0DwkE+Hpnxd6db1tXb0wdeHddG6cA==",
    "license": "Apache-2.0",
    "dependencies": {
      "@smithy/types": "^4.16.1",
      "tslib": "^2.6.2"
    },
    "engines": {
      "node": ">=20.0.0"
    }
  },
  "node_modules/@aws-sdk/util-dynamodb": {
    "version": "3.996.7",
    "resolved": "https://registry.npmjs.org/@aws-sdk/util-dynamodb/-/util-dynamodb-3.996.7.tgz",
    "integrity": "sha512-WJASG9yaW8qNM7pNSgH1PBYz5mVTf7gzKPi0NqGjLlaCtPdk6EjM1lmv03egA07iXUw==",
    "license": "Apache-2.0",
    "dependencies": {
      "tslib": "^2.6.2"
    },
    "engines": {
      "node": ">=20.0.0"
    },
    "peerDependencies": {
      "@aws-sdk/client-dynamodb": "^3.1088.0"
    }
  },
  "node_modules/@aws-sdk/xml-builder": {
    "version": "3.972.36",
    "resolved": "https://registry.npmjs.org/@aws-sdk/xml-builder/-/xml-builder-3.972.36.tgz",

```



```

    "integrity": "sha512-
      RdGmS1GLrtaT0LE1ElSluMldNrpk9Emq6uYs8SS8iHlu5xTAmM9rRkM91o48
      +rIRryBty09t+uLYCoMG6jVMVA==",
    "license": "Apache-2.0",
    "dependencies": {
      "@smithy/types": "^4.16.1",
      "tslib": "^2.6.2"
    },
    "engines": {
      "node": ">=20.0.0"
    }
  },
  "node_modules/@aws/lambda-invoke-store": {
    "version": "0.3.0",
    "resolved": "https://registry.npmjs.org/@aws/lambda-invoke-
      store/-/lambda-invoke-store-0.3.0.tgz",
    "integrity": "sha512-sl4Bm6yiMNYrZKkqqDFWN0UfnWhlS8ivKxrYl
      +6t0gCLrqr8y3B2IqZZbFRkfaVVp7C/baApyh71P+LeE1A2sQ==",
    "license": "Apache-2.0",
    "engines": {
      "node": ">=18.0.0"
    }
  },
  "node_modules/@smithy/core": {
    "version": "3.29.4",
    "resolved": "https://registry.npmjs.org/@smithy/core/-/core
      -3.29.4.tgz",
    "integrity": "sha512-
      G1GRglAabzEhqghJMBAd54FkRS7SAFGHEwbhcI9r+0+
      LIMuFsLyXkLZkCoFSgAglRu8s/URVXJB0hglq3ZipIg==",
    "license": "Apache-2.0",
    "dependencies": {
      "@smithy/types": "^4.16.1",
      "tslib": "^2.6.2"
    },
    "engines": {
      "node": ">=18.0.0"
    }
  },
  "node_modules/@smithy/credential-provider-imds": {
    "version": "4.4.9",
    "resolved": "https://registry.npmjs.org/@smithy/credential-
      provider-imds/-/credential-provider-imds-4.4.9.tgz",
    "integrity": "sha512-2nfV4qRKiYeXU4zD2vvSCfg5dfp/
      BuhrM73vt7q9gzBhxs4rbPxXY21wo+
      kyI3bRmXcEGRnCLTaW80437jzHIg==",
    "license": "Apache-2.0",
    "dependencies": {
      "@smithy/core": "^3.29.4",
      "@smithy/types": "^4.16.1",
      "tslib": "^2.6.2"
    }
  }
}

```

```

    },
    "engines": {
      "node": ">=18.0.0"
    }
  },
  "node_modules/@smithy/fetch-http-handler": {
    "version": "5.6.6",
    "resolved": "https://registry.npmjs.org/@smithy/fetch-http-handler/-/fetch-http-handler-5.6.6.tgz",
    "integrity": "sha512-NHLgAlORUFZjn5ZfhYuyyKMlXA1WLYOdGxEhyNxrPpbJzoacGbl0chn1lN2KiZ8mpNVl
      /OFgw==",
    "license": "Apache-2.0",
    "dependencies": {
      "@smithy/core": "^3.29.4",
      "@smithy/types": "^4.16.1",
      "tslib": "^2.6.2"
    }
  },
  "engines": {
    "node": ">=18.0.0"
  }
},
"node_modules/@smithy/node-http-handler": {
  "version": "4.9.6",
  "resolved": "https://registry.npmjs.org/@smithy/node-http-handler/-/node-http-handler-4.9.6.tgz",
  "integrity": "sha512-odd+
    HYx30LcXRSEz0ZeF3JQdSYdK8QnRgA2N87cPW7coWIbKfRk7a9VQjfeWQLqnzrDLk23
      /JFg==",
  "license": "Apache-2.0",
  "dependencies": {
    "@smithy/core": "^3.29.4",
    "@smithy/types": "^4.16.1",
    "tslib": "^2.6.2"
  }
},
  "engines": {
    "node": ">=18.0.0"
  }
},
"node_modules/@smithy/signature-v4": {
  "version": "5.6.5",
  "resolved": "https://registry.npmjs.org/@smithy/signature-v4/-/signature-v4-5.6.5.tgz",
  "integrity": "sha512-M05VEhwVl0BN7xVoVeNrZfiUFoQtqxUbgl6/
    Rw0TlMMxCSjblG8twSrVTwz3J4w9WZxd2rBfBAUXjH77agspBg==",
  "license": "Apache-2.0",
  "dependencies": {
    "@smithy/core": "^3.29.4",
    "@smithy/types": "^4.16.1",
    "tslib": "^2.6.2"
  }
},

```

```
{
  "engines": {
    "node": ">=18.0.0"
  }
},
{
  "node_modules/@smithy/types": {
    "version": "4.16.1",
    "resolved": "https://registry.npmjs.org/@smithy/types/-/types-4.16.1.tgz",
    "integrity": "sha512-0JFs3V2y2M9tKW5na/qxe69Zv+uxLM07QBbhxF/FHu/Gp2NFZAAL9tWl9PU02xxo07pb3G9FTyjNc6D5uZrJIg==",
    "license": "Apache-2.0",
    "dependencies": {
      "tslib": "^2.6.2"
    }
  },
  "engines": {
    "node": ">=18.0.0"
  }
},
{
  "node_modules/bowser": {
    "version": "2.14.1",
    "resolved": "https://registry.npmjs.org/bowser/-/bowser-2.14.1.tgz",
    "integrity": "sha512-lzB3eTfK2UeEYqXiwjLW33wStMnR5QeHhF3S9OhtYnKCCl3dSi0xVYVwW1klt+C2dtw0F0Ss0UyU4MGs9",
    "license": "MIT"
  },
  "node_modules/mnemonicist": {
    "version": "0.38.3",
    "resolved": "https://registry.npmjs.org/mnemonicist/-/mnemonicist-0.38.3.tgz",
    "integrity": "sha512-2K9QYubXx/NAjv4VLq1d1Ly8pWNC5L3BrixtdkyTegXWJIqY+zLNDhhX/A+ZwWt70tB1S8H4BE8FLYEFyNoOBw==",
    "license": "MIT",
    "dependencies": {
      "obliterator": "^1.6.1"
    }
  },
  "node_modules/obliterator": {
    "version": "1.6.1",
    "resolved": "https://registry.npmjs.org/obliterator/-/obliterator-1.6.1.tgz",
    "integrity": "sha512-SLimUlzuU1hFJUc8zkwyD59Sd+dPOMf05PmnYG/d6Q7HZ+KmkgZJa1PxRso6QdM3sTNHig==",
    "license": "MIT"
  },
  "node_modules/tslib": {
    "version": "2.8.1",

```

```

    "resolved": "https://registry.npmjs.org/tslib/-/tslib
      -2.8.1.tgz",
    "integrity": "sha512-oJFu94HQB+KVduSUQL7wnpmqnfmls0A/
      nAh6b6EH0wCEoK0/mPeXU6c3wKDV83Mk0uHPRHtSXKKU99IBazS/2w
      ==",
    "license": "0BSD"
  }
}
}.
```

package-lock.json

Listing 12: Node.js Dependency Lock File (package-lock.json)

```

{
  "name": "getbooks",
  "version": "1.0.0",
  "lockfileVersion": 3,
  "requires": true,
  "packages": {
    "": {
      "name": "getbooks",
      "version": "1.0.0",
      "dependencies": {
        "@aws-sdk/client-dynamodb": "^3.0.0",
        "@aws-sdk/lib-dynamodb": "^3.0.0"
      }
    },
    "node_modules/@aws-sdk/client-dynamodb": {
      "version": "3.1088.0",
      "resolved": "https://registry.npmjs.org/@aws-sdk/client-
        dynamodb/-/client-dynamodb-3.1088.0.tgz",
      "integrity": "sha512-CVxp88YVq87tW9i2+y6/
        yjtmvVmVt9V6tz2XFBVC2T3zNG/LntIKS1131d321ZX9Bipc9VmKxhJ+
        1+JVb7m/dg==",
      "license": "Apache-2.0",
      "dependencies": {
        "@aws-sdk/core": "^3.975.3",
        "@aws-sdk/credential-provider-node": "^3.972.69",
        "@aws-sdk/dynamodb-protocol": "^3.973.33",
        "@aws-sdk/middleware-endpoint-discovery": "^3.972.25",
        "@aws-sdk/types": "^3.974.2",
        "@smithy/core": "^3.29.4",
        "@smithy/fetch-http-handler": "^5.6.6",
        "@smithy/node-http-handler": "^4.9.6",
        "@smithy/types": "^4.16.1",
        "tslib": "^2.6.2"
      },
      "engines": {
        "node": ">=20.0.0"
      }
    }
  }
}
```

```

},
"node_modules/@aws-sdk/core": {
  "version": "3.975.3",
  "resolved": "https://registry.npmjs.org/@aws-sdk/core/-/core-3.975.3.tgz",
  "integrity": "sha512-ur3kCKuvPLqlsZ2XlvnNBVQ7KkpSu6Y6d0TwSPHLrFpTEfZM8isLBjc4cg96WB7Gif",
  "license": "Apache-2.0",
  "dependencies": {
    "@aws-sdk/types": "^3.974.2",
    "@aws-sdk/xml-builder": "^3.972.36",
    "@aws/lambda-invoke-store": "^0.3.0",
    "@smithy/core": "^3.29.4",
    "@smithy/signature-v4": "^5.6.5",
    "@smithy/types": "^4.16.1",
    "bowser": "^2.11.0",
    "tslib": "^2.6.2"
  },
  "engines": {
    "node": ">=20.0.0"
  }
},
"node_modules/@aws-sdk/credential-provider-env": {
  "version": "3.972.59",
  "resolved": "https://registry.npmjs.org/@aws-sdk/credential-provider-env/-/credential-provider-env-3.972.59.tgz",
  "integrity": "sha512-Ny5e4Mfh3QPmiAc0AiUe+cbTXDlxkU3Rc+EpW0fyWeWey6yp7Fa1KmfNeCc+1a8by9zQ9gtohmiQUkMPScF3ng==",
  "license": "Apache-2.0",
  "dependencies": {
    "@aws-sdk/core": "^3.975.3",
    "@aws-sdk/types": "^3.974.2",
    "@smithy/core": "^3.29.4",
    "@smithy/types": "^4.16.1",
    "tslib": "^2.6.2"
  },
  "engines": {
    "node": ">=20.0.0"
  }
},
"node_modules/@aws-sdk/credential-provider-http": {
  "version": "3.972.61",
  "resolved": "https://registry.npmjs.org/@aws-sdk/credential-provider-http/-/credential-provider-http-3.972.61.tgz",
  "integrity": "sha512-8jAjgStl5Ytq4+HF3X/9f+EmRinaRbGRRtQGktlPfBRVx73H+R1y48vIeXerQtYGFaUqkEp3fT6jP854rV02yQ==",
  "license": "Apache-2.0",
  "dependencies": {
    "@aws-sdk/core": "^3.975.3",

```

```

    "@aws-sdk/types": "^3.974.2",
    "@smithy/core": "^3.29.4",
    "@smithy/fetch-http-handler": "^5.6.6",
    "@smithy/node-http-handler": "^4.9.6",
    "@smithy/types": "^4.16.1",
    "tslib": "^2.6.2"
  },
  "engines": {
    "node": ">=20.0.0"
  }
},
"node_modules/@aws-sdk/credential-provider-ini": {
  "version": "3.973.3",
  "resolved": "https://registry.npmjs.org/@aws-sdk/credential-provider-ini/-/credential-provider-ini-3.973.3.tgz",
  "integrity": "sha512-WpuqYX4gGkx++fCTSWE8+41JzkZVcrI50SH48Ml4CsG1pyuHKyMmpw/FixBHDrmjoQ553PmeCLa/fZlCst+WyA==",
  "license": "Apache-2.0",
  "dependencies": {
    "@aws-sdk/core": "^3.975.3",
    "@aws-sdk/credential-provider-env": "^3.972.59",
    "@aws-sdk/credential-provider-http": "^3.972.61",
    "@aws-sdk/credential-provider-login": "^3.972.65",
    "@aws-sdk/credential-provider-process": "^3.972.59",
    "@aws-sdk/credential-provider-sso": "^3.973.3",
    "@aws-sdk/credential-provider-web-identity": "^3.972.65",
    "@aws-sdk/nested-clients": "^3.997.33",
    "@aws-sdk/types": "^3.974.2",
    "@smithy/core": "^3.29.4",
    "@smithy/credential-provider-imds": "^4.4.9",
    "@smithy/types": "^4.16.1",
    "tslib": "^2.6.2"
  },
  "engines": {
    "node": ">=20.0.0"
  }
},
"node_modules/@aws-sdk/credential-provider-login": {
  "version": "3.972.65",
  "resolved": "https://registry.npmjs.org/@aws-sdk/credential-provider-login/-/credential-provider-login-3.972.65.tgz",
  "integrity": "sha512-xr9rgjYEdmC2Tpg2lwt9o+n0EaK9Qpd+dBjzrVCuWWyQfvh091Ezu0Hh9ts2VUx0ZxmS/k5T9msa34e4R1bnrQ==",
  "license": "Apache-2.0",
  "dependencies": {
    "@aws-sdk/core": "^3.975.3",
    "@aws-sdk/nested-clients": "^3.997.33",
    "@aws-sdk/types": "^3.974.2",

```

```

    "@smithy/core": "^3.29.4",
    "@smithy/types": "^4.16.1",
    "tslib": "^2.6.2"
  },
  "engines": {
    "node": ">=20.0.0"
  }
},
"node_modules/@aws-sdk/credential-provider-node": {
  "version": "3.972.69",
  "resolved": "https://registry.npmjs.org/@aws-sdk/credential-provider-node/-/credential-provider-node-3.972.69.tgz",
  "integrity": "sha512-wbJGGesd0Tl18bmUcbj1xJ+e7CpuRJ6PIpMywLFuUttGy615lua87cJ0EA8pFpY/QgPuUXbnupWBtSPJ9tyZhg==",
  "license": "Apache-2.0",
  "dependencies": {
    "@aws-sdk/credential-provider-env": "^3.972.59",
    "@aws-sdk/credential-provider-http": "^3.972.61",
    "@aws-sdk/credential-provider-ini": "^3.973.3",
    "@aws-sdk/credential-provider-process": "^3.972.59",
    "@aws-sdk/credential-provider-sso": "^3.973.3",
    "@aws-sdk/credential-provider-web-identity": "^3.972.65",
    "@aws-sdk/types": "^3.974.2",
    "@smithy/core": "^3.29.4",
    "@smithy/credential-provider-imds": "^4.4.9",
    "@smithy/types": "^4.16.1",
    "tslib": "^2.6.2"
  },
  "engines": {
    "node": ">=20.0.0"
  }
},
"node_modules/@aws-sdk/credential-provider-process": {
  "version": "3.972.59",
  "resolved": "https://registry.npmjs.org/@aws-sdk/credential-provider-process/-/credential-provider-process-3.972.59.tgz",
  "integrity": "sha512-DlZF2/MhLlatDdlrIy3CUCpfdbLrKx+3SMjVo+WYHnPPwzkc/M3vwAHw40VJf7DMv0+4vfRqSCMc/E9I1auN0g==",
  "license": "Apache-2.0",
  "dependencies": {
    "@aws-sdk/core": "^3.975.3",
    "@aws-sdk/types": "^3.974.2",
    "@smithy/core": "^3.29.4",
    "@smithy/types": "^4.16.1",
    "tslib": "^2.6.2"
  },
  "engines": {
    "node": ">=20.0.0"
  }
}

```

```

},
"node_modules/@aws-sdk/credential-provider-sso": {
  "version": "3.973.3",
  "resolved": "https://registry.npmjs.org/@aws-sdk/credential-provider-sso/-/credential-provider-sso-3.973.3.tgz",
  "integrity": "sha512-hmdDHoy2G5Es2e8IgelNMYUuSQI6uCIAKZMJ2u2PdKDhxxvbk1uWD/g4+R7R5c/tJfKEB1+KjjWiaoCr/S+ZTiQ==",
  "license": "Apache-2.0",
  "dependencies": {
    "@aws-sdk/core": "^3.975.3",
    "@aws-sdk/nested-clients": "^3.997.33",
    "@aws-sdk/token-providers": "3.1088.0",
    "@aws-sdk/types": "^3.974.2",
    "@smithy/core": "^3.29.4",
    "@smithy/types": "^4.16.1",
    "tslib": "^2.6.2"
  },
  "engines": {
    "node": ">=20.0.0"
  }
},
"node_modules/@aws-sdk/credential-provider-web-identity": {
  "version": "3.972.65",
  "resolved": "https://registry.npmjs.org/@aws-sdk/credential-provider-web-identity/-/credential-provider-web-identity-3.972.65.tgz",
  "integrity": "sha512-gHQB/Kt0chjk/JQDa/GJDqmAvEuVn8n7z10wK2h0LFM9TUDRkohg004aEF+s2sBLM0br7Cl5W6P7phgjrrJvLQ==",
  "license": "Apache-2.0",
  "dependencies": {
    "@aws-sdk/core": "^3.975.3",
    "@aws-sdk/nested-clients": "^3.997.33",
    "@aws-sdk/types": "^3.974.2",
    "@smithy/core": "^3.29.4",
    "@smithy/types": "^4.16.1",
    "tslib": "^2.6.2"
  },
  "engines": {
    "node": ">=20.0.0"
  }
},
"node_modules/@aws-sdk/dynamodb-codec": {
  "version": "3.973.33",
  "resolved": "https://registry.npmjs.org/@aws-sdk/dynamodb-codec/-/dynamodb-codec-3.973.33.tgz",
  "integrity": "sha512-awY2mG1zvTy22j4ryZL8jfWQdcuppsw17Mxw1tjG92h0+52JoFZaaTWrJIQp066t/aj4JC4II29UJb6g644vA==",
  "license": "Apache-2.0",

```



```

    "dependencies": {
      "@aws-sdk/core": "^3.975.3",
      "@smithy/core": "^3.29.4",
      "@smithy/types": "^4.16.1",
      "tslib": "^2.6.2"
    },
    "engines": {
      "node": ">=20.0.0"
    }
  },
  "node_modules/@aws-sdk/endpoint-cache": {
    "version": "3.972.9",
    "resolved": "https://registry.npmjs.org/@aws-sdk/endpoint-cache/-/endpoint-cache-3.972.9.tgz",
    "integrity": "sha512-LFvdgq8SriaskUcjpBMDE7J2c9RmuT5v3gU36/zNv71EU5DKUis4FmGFjCMelKCCViFeVrQADBA1Ii0YRhEx6Q==",
    "license": "Apache-2.0",
    "dependencies": {
      "mnemonist": "0.38.3",
      "tslib": "^2.6.2"
    },
    "engines": {
      "node": ">=20.0.0"
    }
  },
  "node_modules/@aws-sdk/lib-dynamodb": {
    "version": "3.1088.0",
    "resolved": "https://registry.npmjs.org/@aws-sdk/lib-dynamodb/-/lib-dynamodb-3.1088.0.tgz",
    "integrity": "sha512-d57jl+BVQWiA1+5NRUtN3/ZKBn0ipQjNuimF3T2LCtA2IcYdCAw37ESDer39ldN4C8BxWRC44RbCmLCkmsx3+w==",
    "license": "Apache-2.0",
    "dependencies": {
      "@aws-sdk/core": "^3.975.3",
      "@aws-sdk/util-dynamodb": "^3.996.7",
      "@smithy/core": "^3.29.4",
      "@smithy/types": "^4.16.1",
      "tslib": "^2.6.2"
    },
    "engines": {
      "node": ">=20.0.0"
    },
    "peerDependencies": {
      "@aws-sdk/client-dynamodb": "^3.1088.0"
    }
  },
  "node_modules/@aws-sdk/middleware-endpoint-discovery": {
    "version": "3.972.25",
    "resolved": "https://registry.npmjs.org/@aws-sdk/middleware-endpoint-discovery/-/middleware-endpoint-discovery

```

```

    -3.972.25.tgz",
    "integrity": "sha512-5G2aVPmbWC5dFI76D0vsNg2L0fgWP4uybcetf
      +06dzeZuRtpihnow88f0l5zm3+ABdIw27FKgyFzV4yB5Bm29Q==",
    "license": "Apache-2.0",
    "dependencies": {
      "@aws-sdk/endpoint-cache": "^3.972.9",
      "@aws-sdk/types": "^3.974.2",
      "@smithy/core": "^3.29.4",
      "@smithy/types": "^4.16.1",
      "tslib": "^2.6.2"
    },
    "engines": {
      "node": ">=20.0.0"
    }
  },
  "node_modules/@aws-sdk/nested-clients": {
    "version": "3.997.33",
    "resolved": "https://registry.npmjs.org/@aws-sdk/nested-
      clients/-/nested-clients-3.997.33.tgz",
    "integrity": "sha512-dVZ0roI/r3/ENvqNGgjMPul+
      jjlZ9GddfVusgTXlVjfZj5isib0xecLkGQbRPp8X0uX+
      RAfjXLFgPkD1JS5xrw==",
    "license": "Apache-2.0",
    "dependencies": {
      "@aws-sdk/core": "^3.975.3",
      "@aws-sdk/signature-v4-multi-region": "^3.996.41",
      "@aws-sdk/types": "^3.974.2",
      "@smithy/core": "^3.29.4",
      "@smithy/fetch-http-handler": "^5.6.6",
      "@smithy/node-http-handler": "^4.9.6",
      "@smithy/types": "^4.16.1",
      "tslib": "^2.6.2"
    },
    "engines": {
      "node": ">=20.0.0"
    }
  },
  "node_modules/@aws-sdk/signature-v4-multi-region": {
    "version": "3.996.41",
    "resolved": "https://registry.npmjs.org/@aws-sdk/signature-
      v4-multi-region/-/signature-v4-multi-region-3.996.41.tgz
      ",
    "integrity": "sha512-QMUytg+FQMGouc8gHS00KoYih3+N6cqmVI/
      pQG0Io7Nr70pQaiXjSY0uL+vsPZ1tymY4LAQ8MYcHJmws5LRxng==",
    "license": "Apache-2.0",
    "dependencies": {
      "@aws-sdk/types": "^3.974.2",
      "@smithy/signature-v4": "^5.6.5",
      "@smithy/types": "^4.16.1",
      "tslib": "^2.6.2"
    }
  },

```

```

    "engines": {
      "node": ">=20.0.0"
    }
  },
  "node_modules/@aws-sdk/token-providers": {
    "version": "3.1088.0",
    "resolved": "https://registry.npmjs.org/@aws-sdk/token-providers/-/token-providers-3.1088.0.tgz",
    "integrity": "sha512-4ObatWt2qpJg5FBk4L00KrTQYzaqeewAtd03r9Z081H9YqLtpTzLyIdyOmJ+nVdfY0nqISkKNfmzP22bNDhwyw==",
    "license": "Apache-2.0",
    "dependencies": {
      "@aws-sdk/core": "^3.975.3",
      "@aws-sdk/nested-clients": "^3.997.33",
      "@aws-sdk/types": "^3.974.2",
      "@smithy/core": "^3.29.4",
      "@smithy/types": "^4.16.1",
      "tslib": "^2.6.2"
    }
  },
  "engines": {
    "node": ">=20.0.0"
  }
},
  "node_modules/@aws-sdk/types": {
    "version": "3.974.2",
    "resolved": "https://registry.npmjs.org/@aws-sdk/types/-/types-3.974.2.tgz",
    "integrity": "sha512-W6IUtSxFbH6X7Wb7DzGCV5QiFQsd0g8b0fntpmDxQlZBoKWUMBu/JPQR0DwkE+Hpnxd6db1tXb0wdeHddG6cA==",
    "license": "Apache-2.0",
    "dependencies": {
      "@smithy/types": "^4.16.1",
      "tslib": "^2.6.2"
    }
  },
  "engines": {
    "node": ">=20.0.0"
  }
},
  "node_modules/@aws-sdk/util-dynamodb": {
    "version": "3.996.7",
    "resolved": "https://registry.npmjs.org/@aws-sdk/util-dynamodb/-/util-dynamodb-3.996.7.tgz",
    "integrity": "sha512-WJASG9yaW8qNM7pNSgH1PBYz5mVTf7gzKPi0NqGjLlaCtPdk6EjM1lmv03egA07iXUw==",
    "license": "Apache-2.0",
    "dependencies": {
      "tslib": "^2.6.2"
    }
  },

```

```

    "engines": {
      "node": ">=20.0.0"
    },
    "peerDependencies": {
      "@aws-sdk/client-dynamodb": "^3.1088.0"
    }
  },
  "node_modules/@aws-sdk/xml-builder": {
    "version": "3.972.36",
    "resolved": "https://registry.npmjs.org/@aws-sdk/xml-builder/-/xml-builder-3.972.36.tgz",
    "integrity": "sha512-RdGmS1GLrtaT0LE1ElSluMldNrpk9Emq6uYs8SS8iHlu5xTAmM9rRkM91o48+rIRryBty09t+uLYCoMG6jVMVA==",
    "license": "Apache-2.0",
    "dependencies": {
      "@smithy/types": "^4.16.1",
      "tslib": "^2.6.2"
    },
    "engines": {
      "node": ">=20.0.0"
    }
  },
  "node_modules/@aws/lambda-invoke-store": {
    "version": "0.3.0",
    "resolved": "https://registry.npmjs.org/@aws/lambda-invoke-store/-/lambda-invoke-store-0.3.0.tgz",
    "integrity": "sha512-sl4Bm6yiMNYrZKkqqDFWN0UfnWhlS8ivKxrYl+6t0gCLrqr8y3B2IqZZbFRkfaVVp7C/baApyh71P+LeE1A2sQ==",
    "license": "Apache-2.0",
    "engines": {
      "node": ">=18.0.0"
    }
  },
  "node_modules/@smithy/core": {
    "version": "3.29.4",
    "resolved": "https://registry.npmjs.org/@smithy/core/-/core-3.29.4.tgz",
    "integrity": "sha512-G1GRglAabzEhqghJMBAd54FkRS7SAFGHEwbhcI9r+0+LIMuFsLyXkLZkCoFSgAg1Ru8s/URVXJB0hglq3ZipIg==",
    "license": "Apache-2.0",
    "dependencies": {
      "@smithy/types": "^4.16.1",
      "tslib": "^2.6.2"
    },
    "engines": {
      "node": ">=18.0.0"
    }
  },
  "node_modules/@smithy/credential-provider-imds": {

```

```

    "version": "4.4.9",
    "resolved": "https://registry.npmjs.org/@smithy/credential-
      provider-imsd/-/credential-provider-imsd-4.4.9.tgz",
    "integrity": "sha512-2nfV4qRKiYeXU4zD2vvSCfg5dfp/
      BuhrM73vt7q9gzBhxs4rbPxXY21wo+
      kyI3bRmXcEGRnCLTaW80437jzHIg==",
    "license": "Apache-2.0",
    "dependencies": {
      "@smithy/core": "^3.29.4",
      "@smithy/types": "^4.16.1",
      "tslib": "^2.6.2"
    },
    "engines": {
      "node": ">=18.0.0"
    }
  },
  "node_modules/@smithy/fetch-http-handler": {
    "version": "5.6.6",
    "resolved": "https://registry.npmjs.org/@smithy/fetch-http-
      handler/-/fetch-http-handler-5.6.6.tgz",
    "integrity": "sha512-
      NHLgA10RUFZjn5ZfhYuyyKM1XA1WLY0dGxEhyNxrPpbJzoacGb10chn11N2KiZ8mpNV
      /OFgw==",
    "license": "Apache-2.0",
    "dependencies": {
      "@smithy/core": "^3.29.4",
      "@smithy/types": "^4.16.1",
      "tslib": "^2.6.2"
    },
    "engines": {
      "node": ">=18.0.0"
    }
  },
  "node_modules/@smithy/node-http-handler": {
    "version": "4.9.6",
    "resolved": "https://registry.npmjs.org/@smithy/node-http-
      handler/-/node-http-handler-4.9.6.tgz",
    "integrity": "sha512-odd+
      HYx30LcXRSEz0ZeF3JQdSYdK8QnRgA2N87cPW7coWIbKfRk7a9VQjfeWQLqnzrDLk23
      /JFg==",
    "license": "Apache-2.0",
    "dependencies": {
      "@smithy/core": "^3.29.4",
      "@smithy/types": "^4.16.1",
      "tslib": "^2.6.2"
    },
    "engines": {
      "node": ">=18.0.0"
    }
  },
  "node_modules/@smithy/signature-v4": {

```

```

    "version": "5.6.5",
    "resolved": "https://registry.npmjs.org/@smithy/signature-
      v4/-/signature-v4-5.6.5.tgz",
    "integrity": "sha512-M05VEhwVl0BN7xVoVeNrZfiUFoQtqxUbg16/
      Rw0TlMMxCSjblG8twSrVTwz3J4w9WZxd2rBfBAUXjH77agspBg==",
    "license": "Apache-2.0",
    "dependencies": {
      "@smithy/core": "^3.29.4",
      "@smithy/types": "^4.16.1",
      "tslib": "^2.6.2"
    },
    "engines": {
      "node": ">=18.0.0"
    }
  },
  "node_modules/@smithy/types": {
    "version": "4.16.1",
    "resolved": "https://registry.npmjs.org/@smithy/types/-/
      types-4.16.1.tgz",
    "integrity": "sha512-0JFs3V2y2M9tKW5na/qxe69Zv+uxLM07QBbhfF
      /FHu/Gp2NFZAAL9tWl9PU02xxo07pb3G9FTyjNc6D5uZrJlIg==",
    "license": "Apache-2.0",
    "dependencies": {
      "tslib": "^2.6.2"
    },
    "engines": {
      "node": ">=18.0.0"
    }
  },
  "node_modules/bowser": {
    "version": "2.14.1",
    "resolved": "https://registry.npmjs.org/bowser/-/bowser
      -2.14.1.tgz",
    "integrity": "sha512-
      tzPjzCxygAKWFOJPO11oxFHs57HzIh0EracIgAePE4pqB3LikALKnSzUyU4MGs9
      /iCEUuHlAJTjTc5M+u7YEGg==",
    "license": "MIT"
  },
  "node_modules/mnemonist": {
    "version": "0.38.3",
    "resolved": "https://registry.npmjs.org/mnemonist/-/
      mnemonist-0.38.3.tgz",
    "integrity": "sha512-2K9QYubXx/
      NAjv4VLq1d1Ly8pWNC5L3BrixtdkyTegXWJIqY+zLNDhhX/A+
      ZwWt70tB1S8H4BE8FLYEFyNo0Bw==",
    "license": "MIT",
    "dependencies": {
      "obliterator": "^1.6.1"
    }
  },
  "node_modules/obliterator": {

```

```
"version": "1.6.1",
"resolved": "https://registry.npmjs.org/obliterator/-/
  obliterator-1.6.1.tgz",
"integrity": "sha512-9WXswnqINnnhOG/5
  SLimUlzuU1hFJUc8zkwyD59Sd+dPOMf05PmnYG/d6Q7HZ+
  KmgkZJa1PxRso6QdM3sTNHig==",
"license": "MIT"
},
"node_modules/tslib": {
  "version": "2.8.1",
  "resolved": "https://registry.npmjs.org/tslib/-/tslib
    -2.8.1.tgz",
  "integrity": "sha512-oJFu94HQB+KVduSUQL7wnpmqnfmls0A/
    nAh6b6EH0wCEoK0/mPeXU6c3wKDV83Mk0uHPRHtSXKKU99IBazS/2w
    ==",
  "license": "0BSD"
}
}
}.
```